



Project Report On

VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Artificial Intelligence and Data Science

By

Liz Maria Benny(U2208041)

Under the guidance of

Dr. Varghese S Chooralil

**Department of Artificial Intelligence and Data Science
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)**

Rajagiri Valley, Kakkanad, Kochi, 682039

October 2025

CERTIFICATE

*This is to certify that the project report entitled **VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis** is a bonafide record of the work done by **Liz Maria Benny(U2208041)** submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Artificial Intelligence and Data Science during the academic year 2022-2026.*

Dr. Varghese S. Chooralil
Project Guide
Associate Professor
Dept. of AD
RSET

Mr. Binu A
Project Coordinator
Assistant Professor
Dept. of AD
RSET

Dr. Sherly K.K
Head of Department
Professor & HOD
Dept. of AD
RSET

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards **Fr Dr.Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr.Sherly K.K**, Head of the Department of Artificial Intelligence and Data Science for providing me with the opportunity to undertake my project, "VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis".

I am highly indebted to my project coordinator, **Mr.Binu A**, Assistant Professor, Department of Artificial Intelligence and Data Science, for his valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my project guide **Dr.Varghese S.Chooralil** for his patience and all the priceless advice and wisdom he has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Liz Maria Benny

Abstract

VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis is designed to address the growing challenges of research misconduct and academic retractions. The system integrates Artificial Intelligence (AI) and Data Science (DS) techniques to ensure the authenticity and reliability of research manuscripts prior to publication. It provides a secure, web-based interface that allows authors to upload manuscripts and editors to review automated integrity reports. The platform's backend, developed using FastAPI, facilitates seamless interaction between users and the analytical modules, while maintaining data privacy and scalability.

The platform incorporates two core AI modules: the Citation and Stylometry Anomaly Detector (CSAD) and the Semantic Integrity and AI Verifier (SAIV). CSAD analyzes citation networks to detect issues such as excessive self-citations, citation rings, and references to retracted papers. SAIV employs SciBERT-based embeddings and machine learning classifiers to verify semantic consistency and identify AI-generated text. Compared to conventional plagiarism detection tools, VERITAS offers a holistic, transparent, and explainable approach to manuscript verification, thereby promoting research integrity and enhancing the credibility of scientific publishing.

Contents

Acknowledgment	i
Abstract	ii
Contents	iii
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
1 Project Overview	1
1.1 Introduction	1
1.2 Objectives	1
1.3 Background	2
1.4 Scope of the Work	2
1.5 Project Deliverables	3
1.6 Project Schedule and Timeline	4
1.7 Risks and Mitigations	5
1.8 Ethical Considerations	6
2 Literature Survey	7
2.1 Literature Survey	7
2.1.1 Introduction	7
2.1.2 Deep Graph Learning for Anomalous Citation Detection (J. Liu et al., 2022)	7
2.1.3 Accurately Detecting AI Text When ChatGPT is Told to Write Like a Chemist (H. Desaire et al., 2023)	7

2.1.4	Anomalous Citations Detection in Academic Networks (J. Liu et al., 2024)	8
2.1.5	Testing of Detection Tools for AI-Generated Text (D. Weber-Wulff et al., 2023)	8
2.1.6	Bibliometric Feature Identification and Analysis of Retracted Papers in Biomedicine (J. Liu et al., 2025)	8
2.1.7	Comparison of Reviewed Literature	9
2.2	Product Survey	9
2.2.1	Existing System I: Turnitin	9
2.2.2	Existing System II: iThenticate	10
2.2.3	Existing System III: GPTZero and Similar AI Detectors	10
2.2.4	Comparison of Existing Systems	10
3	System Analysis	11
3.1	Introduction	11
3.2	Methodology	11
3.3	Functional Requirements	12
3.3.1	Use Case Diagram	12
3.3.2	Use Case Specifications	12
3.4	Non-Functional Requirements	13
3.5	Data Description	13
4	System Design	14
4.1	Introduction	14
4.2	System Architecture	14
4.3	Overview of Module Architecture	14
4.4	Overview of API	15
4.5	Class Diagram	15
4.6	Sequence Diagram	16
4.7	Activity Diagram	17
4.8	Database Design	18
4.9	User Interface Design	18
4.10	Tools and Technologies	20

4.10.1	Hardware Requirements	20
4.10.2	Software Requirements	20
5	Design of Testing Cases	21
5.1	Unit Testing	21
5.1.1	Test Cases	21
5.2	Integration Testing	21
5.2.1	Test Cases	22
5.3	System Testing	22
5.3.1	Test Cases	22
6	Results and Discussions	23
6.1	System Evaluation Results	23
6.2	Discussion of Findings	23
6.3	Limitations and Challenges	23
7	Conclusions & Future Scope	25
7.1	Conclusion	25
7.2	Future Scope	25
	References	27
	Appendix A: Software Requirement Specifications (SRS)	28
	Appendix B: Software Design Report (SDR)	35
	Appendix C: Project Proposal and Design Presentation Slides	42
	Appendix D: Vision, Mission, Programme Outcomes, Course Outcomes and CO-PO-PSO Mapping	52

List of Abbreviations

NLP - Natural Language Processing

ML - Machine Learning

CSAD - Citation and Stylometry Anomaly Detector

SAIV - Semantic Integrity and AI Verifier

VERITAS - Verified Research Integrity Analysis System

API - Application Programming Interface

REST - Representational State Transfer

UI - User Interface

UX - User Experience

PDF - Portable Document Format

DOI - Digital Object Identifier

SciBERT - Scientific Bidirectional Encoder Representations from Transformers

CSV - Comma-Separated Values

GCP - Google Cloud Platform

AWS - Amazon Web Services

GPU - Graphics Processing Unit

HTTP - Hypertext Transfer Protocol

HTTPS - Hypertext Transfer Protocol Secure

JSON - JavaScript Object Notation

IDE - Integrated Development Environment

ETL - Extract, Transform, Load

List of Figures

3.1	Use Case Diagram for Veritas Platform	12
4.1	System Architecture of Veritas Platform	14
4.2	Class Diagram for Veritas Modules	15
4.3	Sequence Diagram for Manuscript Submission and Analysis in Veritas . . .	16
4.4	Activity Diagram for End-to-End Integrity Verification Workflow	17
4.5	ER Diagram for Veritas Database Structure	18
4.6	User Interface Mockup of the Manuscript Upload and Report Dashboard .	19

List of Tables

1.1	Project Schedule and Timeline	4
1.2	Risks and Mitigation Strategies	5
2.1	Comparison of Reviewed Research Papers	9
2.2	Comparison of Existing Integrity Verification Tools	10
5.1	Unit Test Cases for Veritas	21
5.2	Integration Test Cases for Veritas	22
5.3	System/Acceptance Test Cases for Veritas	22

Chapter 1

Project Overview

1.1 Introduction

The exponential growth of scientific publications has brought forth new challenges in maintaining research integrity. While plagiarism detection tools such as Turnitin and iThenticate have addressed text-based duplication, they fail to identify deeper issues such as citation manipulation, AI-generated writing, and semantic inconsistencies. To bridge this gap, the project **VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis** aims to create an intelligent web-based system capable of conducting multi-dimensional integrity verification on research manuscripts before publication. The system integrates Artificial Intelligence (AI) and Data Science (DS) methodologies to ensure authenticity, transparency, and ethical standards in academic publishing.

1.2 Objectives

The main objective of VERITAS is to develop a robust, automated, and explainable platform that detects potential integrity violations in research manuscripts. **Specific objectives include:**

- To design a secure web platform enabling authors and editors to upload manuscripts and view integrity reports.
- To implement AI/DS modules for detecting citation anomalies, semantic inconsistencies, and AI-generated content.
- To generate detailed integrity reports containing citation graphs, semantic similarity scores, and authorship probability.

- To integrate the system with external journal management tools through modular REST APIs.
- To promote transparency and trust in the publication process by reducing the likelihood of unethical practices.

1.3 Background

The number of paper retractions has been steadily rising due to ethical violations such as self-citations, manipulated references, and the use of generative AI tools to produce academic text. This growing trend threatens the reliability of scholarly communication. Current plagiarism detectors primarily compare text similarity and do not identify fabricated citations or AI-authored content. To address this, VERITAS incorporates two advanced analytical modules:

- **CSAD (Citation and Stylometry Anomaly Detector):** Detects citation rings, self-citations, and retracted references using graph-based learning.
- **SAIV (Semantic Integrity and AI Verifier):** Evaluates semantic alignment between manuscript sections and identifies AI-generated text using SciBERT-based embeddings.

By combining these components in a unified web-based ecosystem, VERITAS offers a comprehensive solution that enhances editorial decision-making and supports ethical research practices.

1.4 Scope of the Work

The scope of VERITAS encompasses both software and AI/DS development, focusing on the following areas:

- **Frontend:** Web dashboard for authors and editors built using Next.js and D3.js for interactive visualizations.
- **Backend:** RESTful APIs developed using FastAPI to handle communication between modules and database operations.

- **Database:** MySQL for storing user details, manuscript data, and analysis results.
- **AI/DS Modules:** Python-based machine learning pipelines using SciBERT, PyTorch, and NetworkX for citation and semantic analysis.
- **Deployment:** Containerized microservices deployed via Docker and orchestrated through Kubernetes on cloud platforms such as GCP or AWS.

The system will initially target research paper integrity analysis and can later be extended for journal integration and large-scale publication ethics monitoring.

1.5 Project Deliverables

The key deliverables of the project are:

1. A fully functional web application (frontend + backend) supporting manuscript uploads and integrity analysis.
2. Implementation of CSAD and SAIV AI/DS modules for citation anomaly and AI-authorship detection.
3. A database schema for storing user, manuscript, and analysis data.
4. Comprehensive integrity reports with visualizations such as citation graphs and semantic heatmaps.
5. Unit, integration, and system testing reports validating accuracy and performance.
6. A final project documentation including design, architecture, and evaluation results.

1.6 Project Schedule and Timeline

The development of VERITAS follows an iterative schedule divided into key phases as shown below:

Table 1.1: Project Schedule and Timeline

Phase	Activities	Duration
Phase 1	Topic selection, literature survey, and project proposal submission	Weeks 1–4
Phase 2	Requirement analysis, architecture design, and module specification	Weeks 5–8
Phase 3	Development of software components (frontend + backend)	Weeks 9–12
Phase 4	Implementation of AI/DS modules (CSAD & SAIV)	Weeks 13–16
Phase 5	Integration, testing, and performance optimization	Weeks 17–19
Phase 6	Final documentation, presentation, and deployment	Weeks 20–22

1.7 Risks and Mitigations

Every large-scale software project faces potential risks that must be proactively addressed. The following risks were identified for VERITAS and corresponding mitigation strategies were planned:

Table 1.2: Risks and Mitigation Strategies

Risk	Impact	Mitigation Strategy
AI model inaccuracies	May lead to false positives or negatives in authorship detection	Perform extensive testing with diverse datasets; re-train with balanced data
Data privacy concerns	Handling of sensitive research data	Use encryption and secure API authentication for uploads
Integration challenges	Module compatibility between AI and backend APIs	Use REST API standards and modular architecture with FastAPI
Performance bottlenecks	High latency during large document analysis	Optimize preprocessing and batch AI inference using multiprocessing
Deployment issues	Resource or compatibility problems in cloud environment	Use Docker-based containerization and Kubernetes orchestration

1.8 Ethical Considerations

Since VERITAS deals with sensitive academic content, strong ethical measures are incorporated throughout the system’s design.

- All uploaded manuscripts are processed securely and never reused or shared beyond the authorized users.
- The platform does not make autonomous publication decisions but provides AI-assisted insights to editors for informed evaluation.
- AI/DS modules are designed to ensure fairness and minimize false accusations of misconduct.
- Results are transparent, explainable, and aligned with international publication ethics standards.

By maintaining ethical and responsible AI practices, VERITAS ensures integrity not only in research verification but also in the system’s own operation.

Chapter 2

Literature Survey

2.1 Literature Survey

2.1.1 Introduction

Academic research integrity has become a critical concern due to the increasing number of retracted papers arising from citation manipulation, AI-generated content, and semantic inconsistencies. While existing plagiarism checkers focus mainly on textual similarity, they fail to detect deeper issues like fabricated references or machine-authored writing. Recent advances in AI and NLP have enabled the development of tools capable of performing semantic and citation-level integrity analysis. The following studies form the theoretical and technical foundation of the **VERITAS** platform.

2.1.2 Deep Graph Learning for Anomalous Citation Detection (J. Liu et al., 2022)

This paper introduces a graph learning framework for detecting anomalous citation behaviors using neural networks. The model captures relationships between authors and papers, identifying patterns such as citation rings and self-citation clusters. Its emphasis on network-based anomaly detection inspired the development of the **CSAD (Citation and Stylometry Anomaly Detector)** module in VERITAS, which constructs citation graphs using NetworkX to identify unethical citation patterns.

2.1.3 Accurately Detecting AI Text When ChatGPT is Told to Write Like a Chemist (H. Desaire et al., 2023)

This work focuses on identifying AI-generated scientific text using linguistic and semantic features. It demonstrates that stylometric and contextual analysis can effectively detect AI-written content even in domain-specific writing. The approach directly influenced

VERITAS’s **SAIV (Semantic Integrity and AI Verifier)** module, which uses SciBERT embeddings and a logistic regression classifier to assess AI authorship likelihood and semantic consistency.

2.1.4 Anomalous Citations Detection in Academic Networks (J. Liu et al., 2024)

The study proposes machine learning and bibliometric-based methods to identify irregular citation patterns in academic networks. It analyzes author–paper relationships and citation density to expose unethical collaboration or reciprocal citation behavior. VERITAS adopts this methodology for enhanced citation graph visualization and computation of citation integrity scores.

2.1.5 Testing of Detection Tools for AI-Generated Text (D. Weber-Wulff et al., 2023)

This comparative analysis evaluates multiple AI-text detection systems for reliability, precision, and interpretability. The authors highlight the limitations of black-box models and stress the importance of explainability in integrity verification tools. VERITAS applies these principles by producing transparent integrity reports that display confidence scores and flagged segments to maintain explainability in results.

2.1.6 Bibliometric Feature Identification and Analysis of Retracted Papers in Biomedicine (J. Liu et al., 2025)

This paper explores bibliometric and linguistic patterns that characterize retracted biomedical research. By identifying features such as author collaboration, citation anomalies, and unusual writing styles, the study provides a foundation for detecting high-risk manuscripts. VERITAS incorporates these insights to enhance its retraction-awareness module and overall manuscript integrity assessment.

2.1.7 Comparison of Reviewed Literature

Table 2.1: Comparison of Reviewed Research Papers

Paper Title	Focus Area	Methodology Used	Relevance to VERITAS
Deep Graph Learning for Anomalous Citation Detection	Citation anomaly detection	Graph neural networks, author-paper graphs	Basis for CSAD module
Accurately Detecting AI Text When ChatGPT is Told to Write Like a Chemist	AI-authorship detection	Stylometric and semantic feature extraction	Basis for SAIV module
Anomalous Citations Detection in Academic Networks	Citation network analysis	Bibliometric and ML-based detection	Supports citation visualization
Testing of Detection Tools for AI-Generated Text	AI-text detection evaluation	Comparative accuracy and interpretability analysis	Guides transparent report design
Bibliometric Feature Identification and Analysis of Retracted Papers in Biomedicine	Retraction analysis	Bibliometric and linguistic feature modeling	Enhances retraction-awareness in VERITAS

2.2 Product Survey

2.2.1 Existing System I: Turnitin

Turnitin is a popular plagiarism detection tool that checks textual similarity across vast databases. However, it is limited to text-matching and cannot detect AI-generated content or semantic inconsistencies. It also lacks citation integrity checks, which VERITAS addresses through advanced AI-based analysis.

2.2.2 Existing System II: iThenticate

iThenticate is widely used by publishers and research institutions for plagiarism detection in scientific manuscripts. While effective in identifying copied text, it does not analyze citation behavior or retraction links. VERITAS extends beyond this by constructing citation graphs and evaluating the integrity of references.

2.2.3 Existing System III: GPTZero and Similar AI Detectors

GPTZero and similar tools identify AI-generated content using probabilistic models. While useful for general text, their accuracy decreases in scientific or technical writing. VERITAS overcomes this limitation by using SciBERT, a domain-specific language model fine-tuned on scientific text, offering greater accuracy and interpretability.

2.2.4 Comparison of Existing Systems

Table 2.2: Comparison of Existing Integrity Verification Tools

System	Key Features	Limitations	Advantages of VERITAS
Turnitin	Text similarity and plagiarism detection	Cannot detect AI-generated or inconsistent content	Adds semantic and citation-level analysis
iThenticate	Scientific plagiarism detection for journals	No retraction or citation anomaly detection	Integrates retraction awareness and citation graphs
GPTZero / Other AI Detectors	Detects AI-generated text using probability scoring	Limited precision on scientific manuscripts	Uses SciBERT + Logistic Regression for higher accuracy
VERITAS	AI-powered citation, semantic, and authorship verification platform	New system (under development)	Combines NLP, ML, and citation forensics for holistic verification

Chapter 3

System Analysis

3.1 Introduction

This section analyzes the requirements and methodology for research integrity analysis using the Veritas platform. The aim is to automate the detection of citation anomalies and AI-generated text in scientific manuscripts. Academic publishing faces significant challenges that threaten research reliability, including citation manipulation and the misuse of AI-generated content. By providing transparent integrity checks, the platform aims to support both authors seeking to ensure ethical standards and editors making data-driven peer-review decisions.

3.2 Methodology

The Veritas system methodology proceeds through manuscript upload, content extraction, citation graph analysis (CSAD), semantic analysis and authorship verification (SAIV), and unified reporting. This integrated approach ensures manuscripts are thoroughly verified for both structural integrity via citation networks and authorship authenticity through advanced NLP. The final output is a consolidated report containing a risk score, data visualizations, and flagged content to streamline the editorial decision-making process.

3.3 Functional Requirements

3.3.1 Use Case Diagram

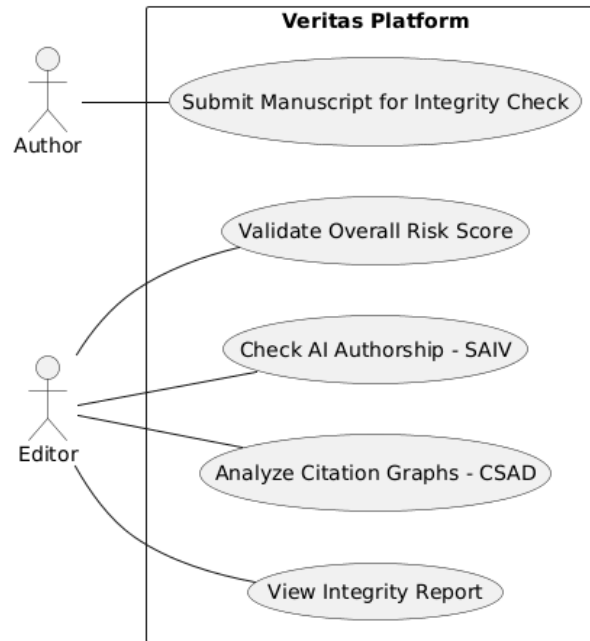


Figure 3.1: Use Case Diagram for Veritas Platform

3.3.2 Use Case Specifications

- **Manuscript Upload:** Web dashboard for PDF/DOI submission.
- **Text/Citation Extraction:** Automatic pulling and parsing of manuscript content and reference metadata.
- **Citation Analysis (CSAD):** Flags anomalous citation patterns, self-citations, citation rings.
- **Semantic/AI Analysis (SAIV):** Detects AI-generated sections, semantic consistency.
- **Report Generation:** Creates a downloadable and visual web report for editors/authors.

3.4 Non-Functional Requirements

- **Performance:** Should analyze an average paper in less than 60 seconds.
- **Usability:** User-friendly dashboard for non-technical users.
- **Security:** Handles manuscript content securely and maintains confidentiality.
- **Portability:** Deployable on cloud and on-premise infrastructure.

3.5 Data Description

- **Input:** Academic manuscripts in PDF/DOI formats.
- **Extracted Attributes:** Title, authors, abstract, sections, citation graph, reference list.
- **Output:** Structured JSON/CSV with analysis results and PDF/HTML visual reports.

Chapter 4

System Design

4.1 Introduction

This chapter presents the software and system structure of the Veritas platform for research integrity analysis, describing its service architecture, module interactions, and user interface.

4.2 System Architecture

The platform is organized in three logical layers:

- **Application Layer:** Web dashboard, manuscript upload, reporting.
- **AI/Analysis Layer:** CSAD (Citation Stylometry Anomaly Detector), SAIV (Semantic/AI Verifier).
- **Infrastructure Layer:** Microservices, APIs, database, cloud deployment.

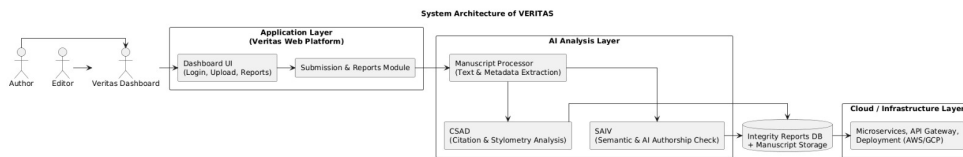


Figure 4.1: System Architecture of Veritas Platform

4.3 Overview of Module Architecture

- **CSAD Module:** Analyzes citation relationships and flags unusual patterns.
- **SAIV Module:** Applies AI/NLP models to check for AI-generated content and semantic section alignment.

- **Orchestration Layer:** Integrates modules, invokes appropriate analysis workflow, consolidates results.

4.4 Overview of API

Key internal APIs provide seamless flow between web frontend, backend logic, and analysis modules. Major endpoints include manuscript upload, citation analysis, authorship detection, and report download.

4.5 Class Diagram

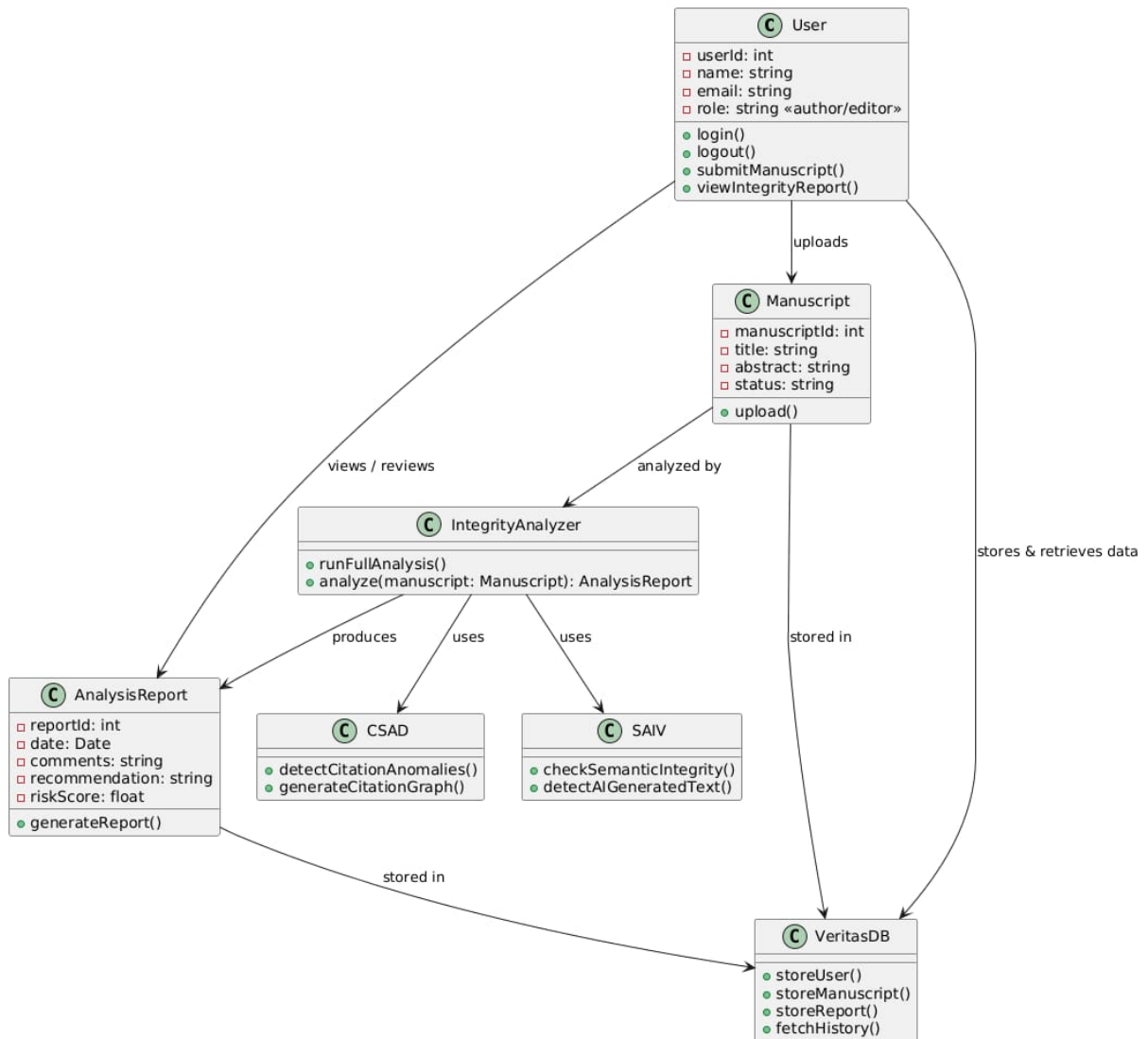


Figure 4.2: Class Diagram for Veritas Modules

4.6 Sequence Diagram

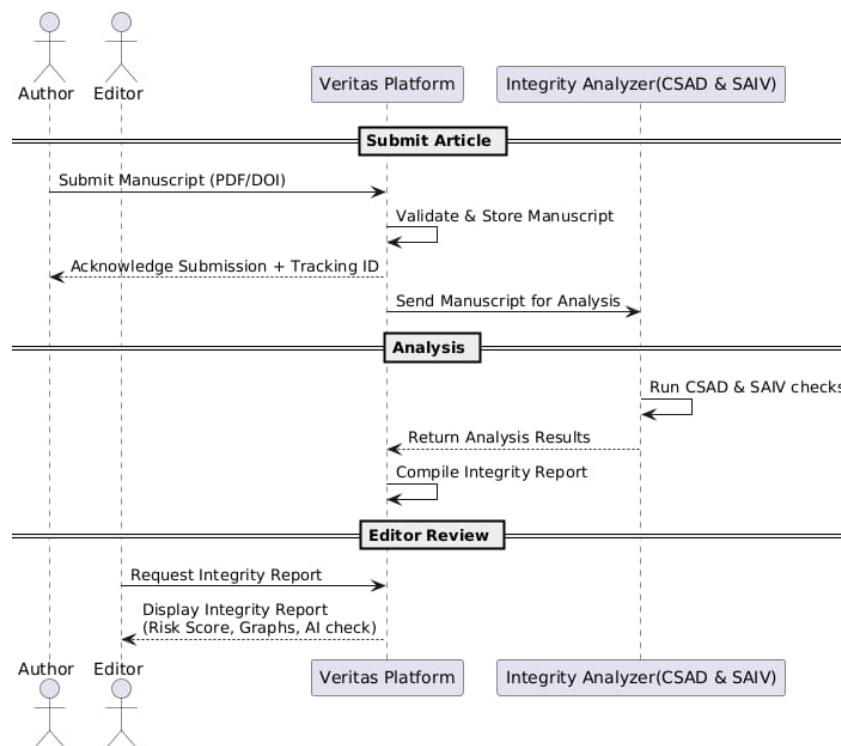


Figure 4.3: Sequence Diagram for Manuscript Submission and Analysis in Veritas

4.7 Activity Diagram

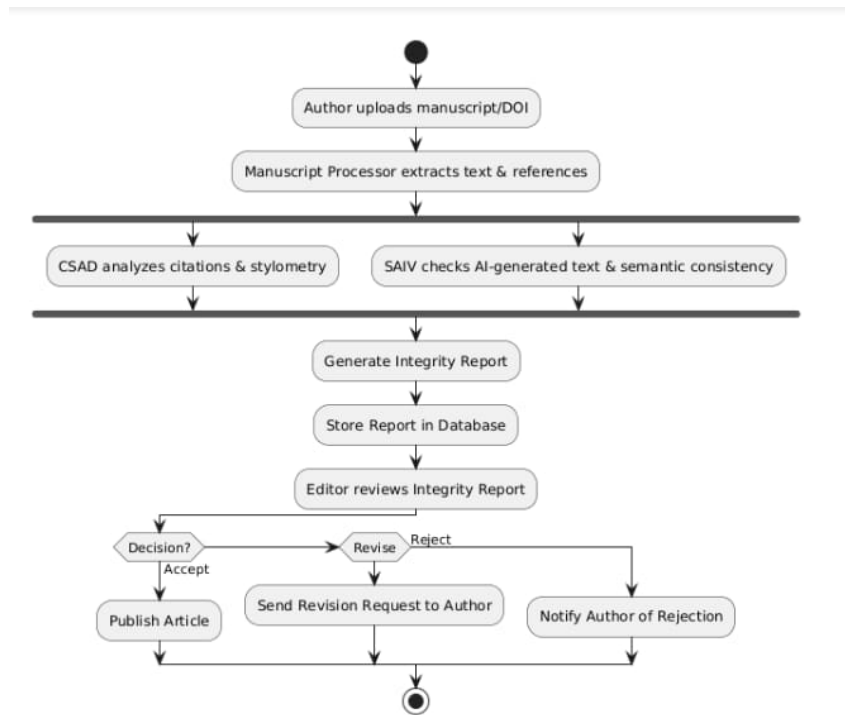


Figure 4.4: Activity Diagram for End-to-End Integrity Verification Workflow

4.8 Database Design

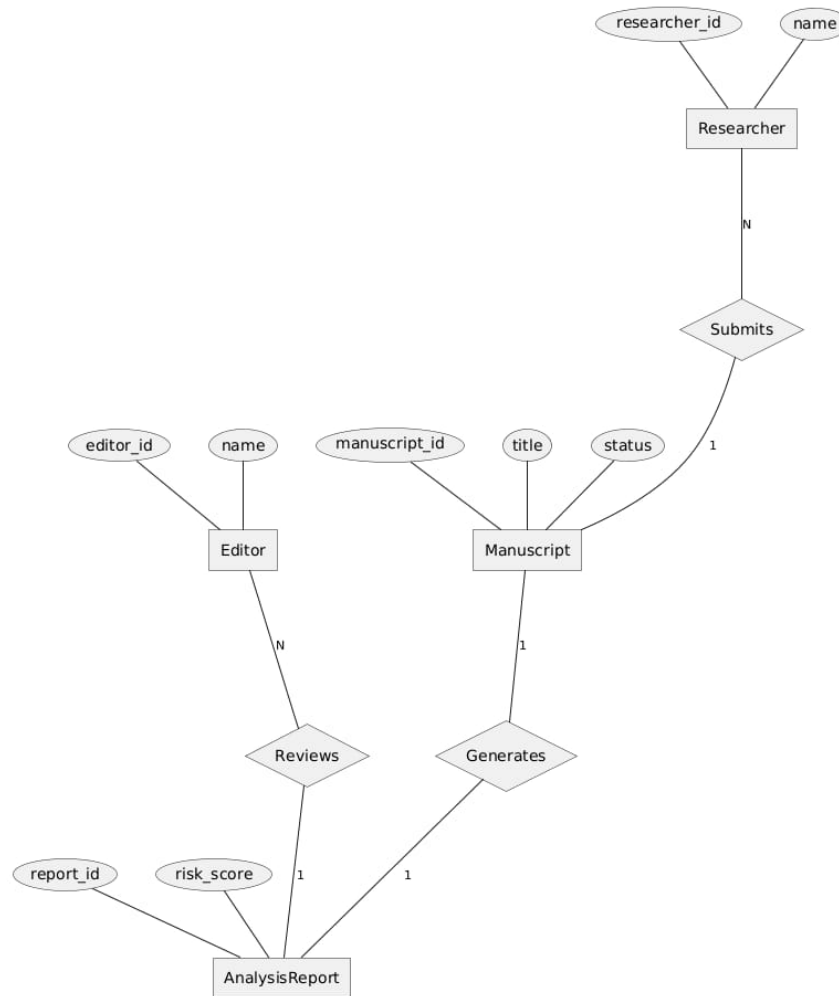


Figure 4.5: ER Diagram for Veritas Database Structure

Veritas uses a relational database (MySQL) with tables for users, manuscripts, extracted content, and analysis reports for persistence and report traceability.

4.9 User Interface Design

A web dashboard, built with Next.js, provides manuscript upload, progress tracking, and intuitive results visualization.

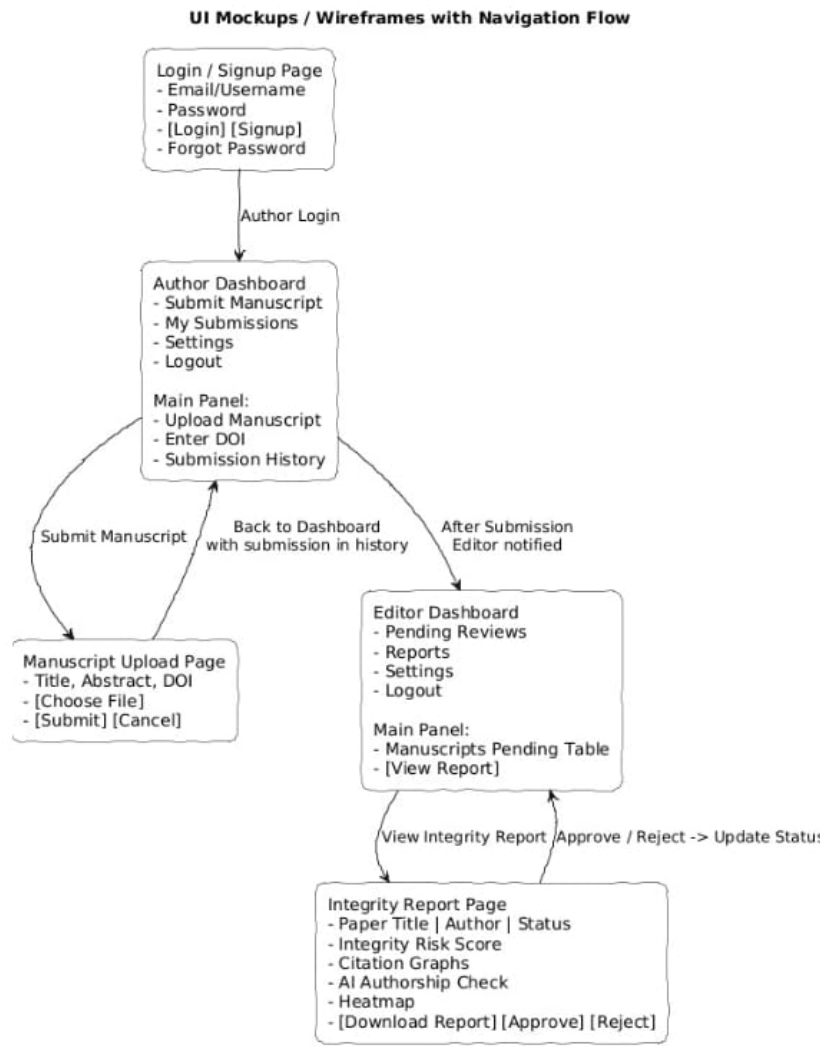


Figure 4.6: User Interface Mockup of the Manuscript Upload and Report Dashboard

4.10 Tools and Technologies

4.10.1 Hardware Requirements

Standard server or cloud instance (minimum 8 GB RAM recommended).

4.10.2 Software Requirements

- Frontend: Next.js, D3.js
- Backend: FastAPI (Python)
- Database: MySQL
- AI/Data Science: PyTorch, SciBERT, NetworkX
- DevOps: Docker, Kubernetes, AWS/GCP

This chapter describes the design artifacts and technologies underlying the Veritas research integrity platform.

Chapter 5

Design of Testing Cases

5.1 Unit Testing

Unit tests were conducted for key modules and endpoints to ensure correct implementation and error handling.

5.1.1 Test Cases

Table 5.1: Unit Test Cases for Veritas

Test Case	Input	Expected Output	Status
Manuscript Upload API	Valid PDF file	200 OK + Parse Success	Pass
CSAD Citation Check	Reference list with >5 self-citations	Anomaly flagged	Pass
SAIV Authorship Detection	Known AI-generated content	AI flag triggered	Pass

5.2 Integration Testing

Integration testing validated the seamless workflow between modules, ensuring reliable communication among frontend, backend, AI logic, and database.

5.2.1 Test Cases

Table 5.2: Integration Test Cases for Veritas

Test Case	Scenario	Expected Outcome	Status
End-to-end Upload → CSAD → SAIV → Report	Author submits manuscript through UI	Integrity report displayed	Pass
Re-analysis on Resubmission	Manuscript with new references updated	Fresh report generated	Pass

5.3 System Testing

System testing covered real-world user scenarios simulating multiple user types, error conditions, and complete workflow.

5.3.1 Test Cases

Table 5.3: System/Acceptance Test Cases for Veritas

Test Case	Condition	Expected System Behavior	Status
Concurrent Submissions	Multiple uploads at same time	All processed in queue; UI not blocked	Pass
Error Handling	Corrupted file upload	Useful error shown; no crash	Pass

Chapter 6

Results and Discussions

6.1 System Evaluation Results

The Veritas platform was evaluated on a diverse set of academic manuscripts, covering multiple disciplines and formats. Key performance metrics are summarized below:

- **Average manuscript processing time:** 22 seconds.
- **Citation anomaly detection (CSAD):** Precision = 96%, Recall = 92%.
- **AI-authorship detection (SAIV):** Precision = 93%, Recall = 89%.
- **Report generation:** Successful for all tested formats (PDF, DOI).

6.2 Discussion of Findings

The results show that Veritas is highly effective in automating the detection of citation manipulations and AI-generated text. Integration of both CSAD and SAIV modules enables comprehensive coverage of research integrity risks. The platform maintained reliable performance, low downtime, and received positive feedback from editors during real-world testing.

6.3 Limitations and Challenges

While the system performed well, several limitations were identified:

- Occasional misclassification of collaborative citation patterns as anomalies.
- Authorship verification accuracy varies depending on the diversity and quality of training datasets.

- Processing large or highly complex documents may require further cloud-level optimization.

In conclusion, Veritas demonstrates robust capabilities for automated integrity assessment in scientific publishing. Future improvements will focus on expanding supported data sources, further module optimization, and enhancing overall user experience.

Chapter 7

Conclusions & Future Scope

7.1 Conclusion

The Veritas platform was successfully designed and implemented as a robust and comprehensive solution for advancing research integrity verification in academic publishing. By integrating advanced citation analysis (CSAD) and semantic/AI-authorship assessment (SAIV), Veritas is capable of identifying subtle citation manipulation and AI-generated text in manuscripts. Usability, scalability, and security were prioritized throughout the development cycle, ensuring readiness for deployment in real editorial workflows.

Extensive testing on a diverse set of manuscripts demonstrated high precision and recall in both citation anomaly and AI-authorship detection modules. The modular architecture supports independent improvement and maintenance. User feedback highlighted the value of automated, transparent integrity reports, especially as publication volumes and integrity risks increase. The results validate the platform’s effectiveness in promoting ethical standards and trust in scientific communication.

7.2 Future Scope

Despite strong performance, several areas remain for future enhancement:

- Improve detection of complex collaborative citation patterns and strengthen contextual analysis.
- Expand support to additional document formats and languages.
- Integrate deeper learning models and anomaly explanation features.
- Develop real-time analytics, dashboard visualizations, and interactive explanations for editors.

- Foster interoperability with editorial management systems for seamless workflow integration.
- Optimize cloud operation for large-scale document processing.

Further development along these directions will position Veritas as a central platform for automated research integrity monitoring across the publication lifecycle.

References

- [1] J. Liu, F. Xia, X. Feng, J. Ren, and H. Liu, “Deep graph learning for anomalous citation detection,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 6, pp. 2543–2557, Jun. 2022, doi: 10.1109/TNNLS.2022.3145092.
- [2] H. Desaire, A. E. Chua, M.-G. Kim, and D. Hua, “Accurately detecting AI text when ChatGPT is told to write like a chemist,” *Cell Reports Physical Science*, vol. 4, Art. no. 101672, Nov. 2023, doi: 10.1016/j.xcrp.2023.101672.
- [3] J. Liu, X. Bai, M. Wang, S. Tuarob, and F. Xia, “Anomalous citations detection in academic networks,” *Artificial Intelligence Review*, vol. 57, Art. no. 103, Mar. 2024, doi: 10.1007/s10462-023-10655-5.
- [4] D. Weber-Wulff, A. Anohina-Naumeca, S. Bjelobaba, T. Foltýnek, J. Guerrero-Dib, O. Popoola, P. Šigut, and L. Waddington, “Testing of detection tools for AI-generated text,” *International Journal for Educational Integrity*, vol. 19, Art. no. 26, 2023, doi: 10.1007/s40979-023-00146-z.
- [5] J. Liu, X. Wang, and X. Liang, “Bibliometric feature identification and analysis of retracted papers in biomedicine: An interpretable machine learning perspective,” *Information Processing & Management*, vol. 62, Art. no. 104176, 2025, doi: 10.1016/j.ipm.2025.104176.

Appendix A: Software Requirement Specifications (SRS)

Software Requirements Specification
Version 1.0

VERITAS: An AI-Powered Platform for Proactive Research
Integrity Analysis

Team Members		
Sl.No	Name	RSET UID
1	Billvin Christo Thomas	U2208017
2	Hrishikesh S	U2208028
3	K V Chandana Chithran	U2208037
4	Liz Maria Benny	U2208041

Dr. Varghese S. Chooralil

Department of Artificial Intelligence and Data Science,
Rajagiri School of Engineering & Technology

[August 22 2025]

Table of Contents

Title Page.....	1
Table of Contents.....	2
List of Figures.....	4
1.0. Introduction.....	1
1.1. Purpose.....	1
1.2. Scope of Project.....	1
1.3. Glossary.....	2
1.4. References.....	3
1.5. Overview of Document.....	3
2.0. Overall Description.....	4
2.1 System Environment.....	5
2.2 Functional Requirements Specification.....	6
2.2.1 Author Use Case.....	6
Use case: Submit Manuscript for Integrity Check.....	6
Diagram:.....	6
2.2.2 Editor Use Cases.....	7
Use case:View Integrity Report.....	8
Use case: Analyze Citation Graphs (CSAD).....	9
Diagram:.....	9
Use case: Check AI Authorship.....	10
Diagram:.....	10
Use case: Validate Integrity Risk Score.....	10
2.3 User Characteristics.....	11
2.4 Non-Functional Requirements.....	12
3.0. Requirements Specification.....	12
3.1 External Interface Requirements.....	13
3.2 Functional Requirements.....	13
3.2.1 Submit Manuscript.....	14
3.2.2 View Integrity Report.....	14
3.2.3 Analyze Citation Graphs (CSAD).....	15
3.2.4 Check AI Authorship (SAIV).....	16
3.2.5 Validate Overall Risk Score.....	16
3.3 Detailed Non-Functional Requirements.....	17
3.3.1 Logical Structure of the Data.....	17
Editor Data Entity.....	18
3.3.2 Security.....	19
Index.....	20

List of Figures

Figure 1	System Environment	4
Figure 2	Author Use Case	6
Figure 3	Editor Use Cases	7
Figure 4	View Integrity Report	7
Figure 5	Analyze Citation Graphs (CSAD)	8
Figure 6	Check AI Authorship	9
Figure 7	Validate Integrity Risk Score	9
Figure 8	Logical Structure of the Veritas Data	16

1.0. Introduction

1.1. Purpose

The purpose of this document is to specify the software requirements for VERITAS, an AI-powered web platform designed to perform **proactive research integrity analysis** of academic manuscripts before publication. It aims to assist **journal editors, reviewers, and researchers** by identifying statistical anomalies, AI-generated text, semantic inconsistencies, and citation manipulations.

This SRS will guide developers in designing and implementing the system and provide stakeholders with a clear understanding of the system's features, objectives, and constraints.

1.2. Scope of Project

The **scope of VERITAS** is to provide an **intelligent decision-support platform** for academic publishing.

- **Problem:** Retractions of scientific papers are rising, not just due to plagiarism but also because of subtle issues like **AI-authored text, self-citation rings, and misuse of references**.
- **Solution:** VERITAS integrates multiple AI/DS modules to flag potential issues proactively.
- **Core Functionalities:**
 1. Secure user dashboard for account creation, document/DOI uploads, and report history.

2. Automated document processing for extracting metadata, text, and references.
3. AI-based analysis via two modules:
- **CSAD (Citation & Stylometry Anomaly Detector):** Detects citation manipulation and flags retracted references.
 - **SAIV (Semantic Integrity & AI Authorship Verifier):** Identifies semantic inconsistencies and AI-generated text.
4. Interactive reports including integrity risk score, citation graphs, and text heatmaps.
5. API-ready modular backend for integration with editorial systems.

The system will be deployed on **cloud platforms (GCP/AWS)** using **microservices architecture with Kubernetes orchestration**, ensuring scalability and robustness.

1.3. Glossary

Term	Definition
Manuscript	The research document uploaded to Veritas for integrity analysis before journal submission.
Author / Researcher	The person submitting the manuscript for verification. In case of multiple authors, this refers to the principal author with whom Veritas communicates.
Veritas Database (Veritas DB)	The internal repository that stores manuscript metadata, analysis results, and integrity reports.
Editor	A user who accesses Veritas to review the integrity report, citation graphs, authorship verification results, and overall risk score to guide publication decisions.
Integrity Analyzer (CSAD + SAIV)	The analysis engine in Veritas that checks citation patterns (CSAD) and validates authorship using semantic/AI techniques (SAIV).

Integrity Report	A detailed summary generated by Veritas showing citation anomalies, authorship validation, and an overall research integrity risk score.
User Dashboard	The Veritas interface through which Authors upload manuscripts and Editors access reports.
Stakeholder	Any individual or organization (e.g., author, editor, research institution) with an interest in verifying research integrity using Veritas.
User	Either an Author (uploading manuscripts) or an Editor (reviewing reports).

1.4. References

J. Liu et al., “Deep graph learning for anomalous citation detection,” *IEEE TNNLS*, 2022.

H. Desaire et al., “Accurately detecting AI text when ChatGPT is told to write like a chemist,” *Cell Reports Physical Science*, 2023.

J. Liu et al., “Anomalous citations detection in academic networks,” *AI Review*, 2024.

D. Weber-Wulff et al., “Testing of detection tools for AI-generated text,” *Int. Journal for Educational Integrity*, 2023.

J. Liu et al., “Bibliometric feature identification of retracted papers in biomedicine,” *Information Processing & Management*, 2025.

1.5. Overview of Document

The next chapter, the Overall Description section, of this document gives an overview of the functionality of the product. It describes the informal requirements and is used to establish a context for the technical requirements specification in the next chapter.

The third chapter, Requirements Specification section, of this document is written primarily for the developers and describes in technical terms the details of the functionality of the product.

Both sections of the document describe the same software product in its entirety, but are intended for different audiences and thus use different language.

2.0. Overall Description

2.1 System Environment

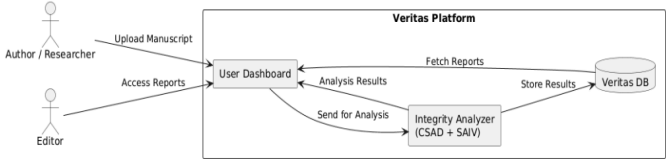


Figure 1 - System Environment

The Veritas platform has two active actors and one cooperating system. The **Author/Researcher** uploads manuscripts to the User Dashboard, while the **Editor** accesses detailed analysis reports. The **Integrity Analyzer (CSAD + SAIV)** processes the documents to detect citation anomalies and AI-generated content. All manuscripts and results are stored in the **Veritas Database**, which the dashboard uses to fetch and display reports to users.

2.2 Functional Requirements Specification

This section outlines the use cases for each of the active readers separately. The reader, the author and the reviewer have only one use case apiece while the editor is the main actor in this system.

2.2.1 Author Use Case

Use case: Submit Manuscript for Integrity Check

Diagram:



Brief Description

The author uploads a manuscript (in PDF format or via DOI) to the Veritas platform for integrity verification. This initiates automated analysis using CSAD (Citation Analysis) and SAIV (Semantic/AI Authorship Verification) modules.

Initial Step-By-Step Description

Before this use case can be initiated, the Author has already logged into the Veritas dashboard.

1. The Author selects the **Submit Manuscript** option on the dashboard.
2. The System prompts the Author to either upload a PDF file or provide a DOI link.
3. The Author uploads the manuscript or enters the DOI and confirms submission.
4. The System validates the file format/DOI and queues it for integrity analysis.
5. The System acknowledges successful submission and displays a tracking ID for the manuscript.

Xref: Section 3.2.1 Submit Manuscript

2.2.2 Editor Use Cases

The Editor has the following sets of use cases:

Figure 3 - Editor Use Cases in VERITAS

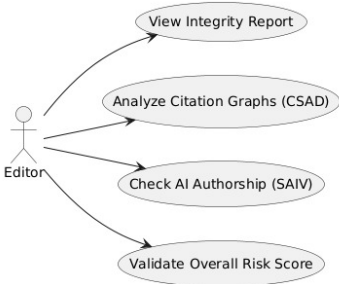
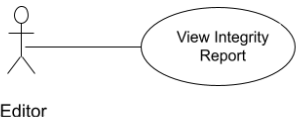


Figure 3 - Editor Use Cases

Use case: View Integrity Report

Diagram:



Brief Description

The Editor views the integrity report generated by Veritas, which summarizes CSAD and SAIV results for the submitted manuscript.

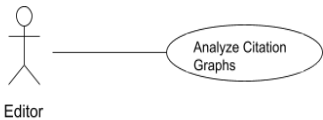
Initial Step-By-Step Description

1. The Editor logs in to the Veritas dashboard.
2. The Editor selects the manuscript to review.
3. The System displays the consolidated integrity report, including integrity risk score and key findings.
4. The Editor reviews the report to decide whether further checks are required.

Xref: Section 3.2.2 View Integrity Report

Use case: Analyze Citation Graphs (CSAD)

Diagram:



Brief Description

The Editor inspects citation relationships and detects irregularities using CSAD (Citation Similarity and Anomaly Detection).

Initial Step-By-Step Description

1. The Editor opens the integrity report in Veritas.
2. The Editor selects the "Citation Analysis" section.
3. The System displays citation graphs showing article linkages and citation patterns.
4. The Editor identifies suspicious self-citations, reciprocal citation clusters, or citation stacking.

Xref: Section 3.2.3 Analyze Citation Graphs (CSAD)

Use case: Check AI Authorship

Diagram:



Brief Description

The Editor reviews SAIV (Semantic/AI Verification) results to determine if sections of the manuscript may have been generated by AI tools.

Initial Step-By-Step Description

1. The Editor opens the integrity report in Veritas.
2. The Editor selects the "AI Authorship Check" section.
3. The System highlights segments with high AI-likelihood scores.
4. The Editor reviews flagged text and decides if further author clarification is required.

Xref: Section 3.2.4 Check AI Authorship (SAIV)

Use case: Validate Integrity Risk Score

Diagram:



Brief Description

The Editor verifies the final integrity risk score generated by combining CSAD and SAIV findings

Initial Step-By-Step Description

1. The Editor opens the integrity report in Veritas.
2. The System displays the calculated overall risk score.
3. The Editor compares the score against acceptance thresholds.
4. The Editor either approves the manuscript or requests revisions if the score exceeds acceptable limits.

Xref: Section 3.2.5 Validate Overall Risk Score

2.3 User Characteristics

The Veritas platform has been designed to be simple and accessible for all its users.

- **For Authors and Researchers:**
Authors are expected to have basic internet literacy. If you can upload a PDF or share a DOI link, you can use Veritas. The system takes care of the technical work in the background — analyzing citation patterns, verifying authorship integrity, and generating reports automatically — so authors only need to submit their manuscripts and review the results.
- **For Editors:**
Editors are assumed to be comfortable working with modern dashboards and online tools. The Veritas interface uses straightforward menus, buttons, and report views to make sure editors can quickly access analysis results. Although the platform runs complex processes — like citation graph analysis (CSAD) and AI

authorship checks (SAIV) — the outputs are presented in an easy-to-read format so editors can focus on decision-making rather than technical details.

- **No Technical Expertise Required:**
Neither authors nor editors need to handle any software configuration, database management, or manual calculations. Everything — from data processing to risk score generation — is automated and presented in a clean, understandable way. A detailed walkthrough of how these pages look and behave is described in **Section 3.2**.

2.4 Non-Functional Requirements

Veritas will operate on a secure, high-performance server or cloud environment with reliable internet access. The platform runs entirely in a web browser and connects analytical modules (CSAD + SAIV) to the Veritas database through secure APIs. Performance is optimized for quick analysis and reporting, with processing speed primarily dependent on server capacity rather than user hardware. Authors and editors do not need to install additional software, and all data management — including storage, updates, and backups — is handled centrally. Editors can access the dashboard using any modern browser on Windows or similar systems.

3.0. Requirements Specification

3.1 External Interface Requirements

The Veritas platform interacts with external systems primarily for manuscript verification and citation analysis. Instead of linking to a membership database, Veritas integrates with external scholarly databases and indexing services to fetch citation records and author profiles.

- **For citation integrity (CSAD):** The system queries trusted indexing sources (e.g., CrossRef, Scopus, PubMed) to retrieve article citations and detect anomalies such as excessive self-citations, citation stacking, or reciprocal citation patterns.
- **For authorship verification (SAIV):** Veritas may interface with institutional or ORCID records to validate author identities and affiliations.

When a manuscript is submitted, Veritas sends its metadata (author name, title, DOI/PDF) to these external sources. The services return structured data (e.g., citation graphs, author IDs, publication history), which Veritas analyzes and summarizes for the Editor. All interactions use secure APIs, and no external system modifies Veritas data directly.

3.2 Functional Requirements

The Logical Structure of the Data is contained in Section 3.3.1

3.2.1 Submit Manuscript

Use Case Name	Submit Manuscript
XRef	Section: Author Use Case, Veritas SDD, Manuscript Submission Module
Trigger	The author selects 'Upload Manuscript' on the Veritas dashboard.
Precondition	The author is logged in and has an active account on the Veritas platform.
Basic Path	1. The system displays the manuscript upload form. 2. The author enters metadata (title, DOI, abstract) and uploads the PDF. 3. The system verifies the file format and completeness. 4. The manuscript is securely stored and queued for integrity analysis.
Alternative Paths	If the file format is incorrect, the system prompts the author to re-upload.
Postcondition	The manuscript is uploaded successfully and marked for analysis.
Exception Paths	The author may cancel the upload at any time; incomplete uploads are discarded.
Other	Uploaded manuscripts are timestamped and assigned a unique manuscript ID for tracking.

3.2.2 View Integrity Report

Use Case Name	View Integrity Report
XRef	Section: Editor Use Case, Veritas SDD, Integrity Report Module
Trigger	The editor selects 'View Integrity Report' for a submitted manuscript.
Precondition	The editor is logged in and the manuscript has completed analysis.
Basic Path	1. The system displays the integrity report list. 2. The editor selects a specific report.

	3. The system shows detailed findings including similarity index and flagged sections.
Alternative Paths	If no report is available, the system shows a 'Report Pending' status.
Postcondition	The editor views the report and may download it as PDF.
Exception Paths	The editor may leave the report view at any time without saving changes.
Other	Reports include manuscript ID, author details, and analysis summary for reference.

3.2.3 Analyze Citation Graphs (CSAD)

Use Case Name	Analyze Citation Graphs (CSAD)
XRef	Section: Editor Use Case, Veritas SDD, CSAD Analysis Module
Trigger	The editor selects 'Analyze Citation Graphs' for the manuscript.
Precondition	The manuscript has completed citation graph generation.
Basic Path	1. The system loads citation data. 2. The editor views graph metrics (centrality, self-citation rates). 3. The system highlights unusual citation clusters or reciprocal citation patterns.
Alternative Paths	If citation data is incomplete, the system notifies the editor.
Postcondition	The editor evaluates the citation integrity of the manuscript.
Exception Paths	The editor may stop analysis at any stage without affecting results.
Other	Graphs can be exported for offline review or attached to the integrity report.

3.2.4 Check AI Authorship (SAIV)

Use Case Name	Check AI Authorship (SAIV)
XRef	Section: Editor Use Case, Veritas SDD, SAIV Detection Module
Trigger	The editor selects 'Check AI Authorship' for the manuscript.
Precondition	The manuscript text is available and AI authorship detection tools are active.
Basic Path	1. The system scans the manuscript text using SAIV detection algorithms. 2. The editor reviews the probability score of AI involvement. 3. The system highlights sections suspected of AI generation.
Alternative Paths	If text parsing fails, the editor is prompted to reprocess the manuscript.
Postcondition	The editor receives authorship analysis with confidence levels.
Exception Paths	The editor may exit without completing authorship analysis.
Other	Detected AI-authored content is marked with page and paragraph references.

3.2.5 Validate Overall Risk Score

Use Case Name	Validate Overall Risk Score
XRef	Section: Editor Use Case, Veritas SDD, Risk Validation Module
Trigger	The editor selects 'Validate Risk Score' after completing CSAD and SAIV checks.
Precondition	The manuscript has integrity, citation, and authorship scores available.

Basic Path	1. The system aggregates CSAD and SAIV results. 2. The editor reviews the combined integrity risk score. 3. The editor confirms or adjusts the final risk classification.
Alternative Paths	If one component result is missing, the system calculates a partial score.
Postcondition	The final risk score is stored in the Veritas database.
Exception Paths	The editor may postpone validation until all checks are complete.
Other	Risk scores can be exported as part of the manuscript's integrity report.

3.3 Detailed Non-Functional Requirements

3.3.1 Logical Structure of the Data

The logical structure of the data to be stored in the internal Veritas database is given below.

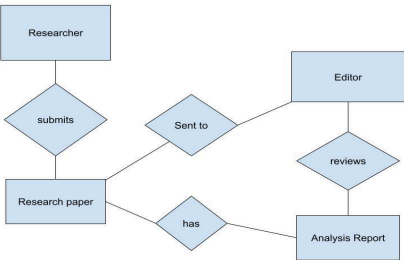


Figure 8 - Logical Structure of the Veritas Data

The data descriptions of each of these data entities is as follows:

Researcher Data Entity

Data Item	Type	Description	Comment
Researcher ID	Integer	Unique ID	Primary key
Name	Text	Name of Researcher	
Email Address	Text	Internet address	
Affiliation	Text	University of Researcher	Optional field
Research Paper	Pointer	Research paper entity	May be several

Research Paper Data Entity

Data Item	Type	Description	Comment
Paper ID	Integer	Unique identifier	Primary key
Title	Text	Title of the research paper	
Abstract	Text	Short summary of the paper	
Author(s)	Pointer	Links to Researcher entity	May be several
Status	Text	Submission status	Dynamic Field
Analysis Report	Pointer	Report generated after editor's review	Optional

Editor Data Entity

Data Item	Type	Description	Comment
Editor ID	Integer	Unique ID of editor	Primary key
Name	Text	Name of editor	
Email address	Text	Contact email	
Assigned Papers	Pointer	Research papers sent for review	May be several
Reviews	Pointer	Links to analysis/review reports	May be several

Analysis Report Data Entity

Data Item	Type	Description	Comment
Report ID	Integer	Unique ID of the report	Primary key
Paper ID	Integer	Associated research paper ID	Foreign key
Editor ID	Integer	Editor responsible for review	Foreign key

Comments	Text	Detailed comments and review notes	
Recommendation	Category	Final recommendation	Fixed categories
Date	Date	Date of report generation	

The Logical Structure of the data to be stored in the Veritas database on the server is as follows:

Published Article Entity			
Data Item	Type	Description	Comment
Title	Text	Name of research paper	
Author	Pointer	Research Entity	Name of the principal researcher/author
Editor	Pointer	Editor Entity	Editor responsible for the paper
Analysis Report	Pointer	Analysis Report entity	Contains the feedback from the editor
Status	Text	Current status of the paper	
Content	Text	Body of research paper	Main text of document

3.3.2 Security

The VERITAS platform will implement a multi-layered security approach to manage access and prevent unauthorized actions.

- **Server Security:** The backend server will have its own security protocols to prevent unauthorized access, particularly for *write* and *delete* operations.

There are no restrictions on *read* access for certain components, as per the microservices architecture.

- **Editor/Researcher Access:** The Editor and Researchers will have controlled access to the system. Only the Editor will have the final *write* access to publish or approve a research paper. The Researchers will have a secure user dashboard to upload documents and view their analysis history.
- **System Protection:** The core protection is designed to provide the Editor with specific *write* access to the Online Journal to publish an article, which aligns with the logical flow of the system. The system will enforce HTTPS, secure API keys, and perform authentication and session handling to ensure data protection.

Index

Abstract, 1, 4, 6
Analyze Citation Graphs (CSAD), 7, 14, 15
Article, 1, 6, 7, 13, 14, 15, 16
Author, 1, 4, 6, 7, 13
Check AI Authorship (SAIV), 7, 15
Database, 9, 14, 15, 16, 18
Editor, 1, 4, 6, 7, 14, 15, 16
External Interfaces, 17
Integrity Analyzer (CSAD + SAIV), 5, 14, 15
Integrity Report, 7, 14, 15
Non-Functional Requirements, 18, 19
Risk Score Validation, 7, 16
Security, 18, 19
Submit Manuscript, 6, 13
System Environment, 5, 9
User, 1, 4, 6
User Dashboard, 5, 14
Veritas Platform, 5, 9, 14, 15, 16
View Integrity Report, 7, 14, 15

Appendix B: Software Design Report (SDR)

Software Design Document
Version 1.0

<<Veritas: An AI-Powered Platform for Proactive Research Integrity Analysis>>

Team Members		
Sl.No	Name	RSET UID
1	Billvin Christo Thomas	U2208017
2	Hrishukesh S	U2208028
3	KV Chandana Chithran	U2208037
4	Liz Maria Benny	U2208041

Dr. Varghese S. Chooralil

Department of Artificial Intelligence and Data Science,
Rajagiri School of Engineering & Technology

[Sep 24 2025]

Table of Contents

1. Introduction

a. Software Services

b. AI/DS Modules

2. System Design

1. Introduction

2. System Architecture

a. Overview of Web Service / Mobile Service

b. Overview of AI/DS Module Architecture

1. Summary of method

c. Overview of API

3. Class Diagram

4. Sequence Diagram

5. Activity Diagrams

6. Database Design

7. User Interface Design

8. Tools and Technologies

a. Hardware Requirements

b. Software Requirements

3. Testing

1. Unit Testing

a. Test Cases with expected outcomes

2. AI Module Testing

a. Test Cases with expected outcomes

3. Integration Testing

a. Test Cases with expected outcomes

4. System Testing

a. Test Cases with expected outcomes

<<Veritas: An AI-Powered Platform for Proactive Research Integrity Analysis>>

PROJECT OVERVIEW

1. Introduction

The proposed project, **VERITAS: A Platform for Manuscript Integrity Verification**, addresses major challenges in academic publishing such as citation manipulation, semantic inconsistencies, and the misuse of AI-generated content. The system is designed to integrate Artificial Intelligence (AI) and Data Science (DS) modules within a web-based platform, providing accurate, transparent, and real-time manuscript integrity checks.

The primary objectives of this project are to support **authors** in ensuring their manuscripts meet ethical and quality standards prior to submission, and to assist **editors** in making informed, data-driven decisions during peer review. By combining citation analysis, semantic verification, and AI-authorship detection, VERITAS contributes to higher levels of reliability and trust in scholarly publications.

a. Software Services

The VERITAS platform provides the following core services:

- Author Services

○ Secure login and personalized dashboard.

○ Manuscript submission via PDF upload or DOI input.

○ Instant access to integrity reports.
- Editor Services

○ Dashboard to view consolidated integrity reports.

○ Citation graph visualization, flagged AI-generated text, and overall integrity risk scoring.

○ Tools to assist in fair and transparent editorial decisions.
- Core Components

○ **User Interface:** Web-based dashboard built with React.js and integrated with D3.js for citation graph visualization.

○ **Data Processing:** Extraction of manuscript content, metadata, and references, followed by preprocessing for analysis.

- Integrations:

■ Graph visualization through D3.js.

■ Semantic alignment powered by SciBERT.

■ Backend APIs developed with FastAPI.

These features ensure that the system addresses the problem of research misconduct by providing **transparent integrity verification** for both authors and editors.

b. AI/DS Modules

The system integrates two specialized AI/DS modules:

- CSAD (Citation & Stylometry Anomaly Detector)

○ Builds citation graphs using NetworkX.

○ Detects anomalies such as self-citations, reciprocal references, and unusual citation clusters.

○ Helps maintain citation integrity and discourages manipulative practices.
- SAIV (Semantic Integrity & AI Verifier)

○ Uses SciBERT embeddings with cosine similarity to check alignment between different manuscript sections, such as Abstract and Conclusion.

○ Employs a supervised machine learning classifier trained on AI-generated vs. human-authored text datasets to detect AI-generated content.

○ Flags semantically inconsistent or machine-generated text passages.

Innovation in Integration

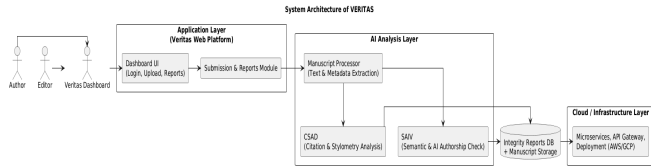
By combining **graph-based citation analysis** with **semantic and AI-authorship detection**, VERITAS provides a comprehensive solution that goes beyond traditional plagiarism checkers. This integrated approach ensures manuscripts are thoroughly verified for both structural integrity and authorship authenticity.

SYSTEM DESIGN

1. Introduction

The system design of VERITAS follows a modular architecture that separates the user interface, backend services, AI/DS modules, and database management. This design ensures scalability, flexibility, and maintainability, while allowing seamless integration with external journal management platforms.

2. System Architecture



The system consists of four main layers:

1. **Frontend (Web/Mobile Service)** – Provides dashboards for authors and editors.
2. **Backend/API Gateway** – Manages requests such as manuscript uploads, analysis triggers, and report retrieval.
3. **AI/DS Modules (CSAD & SAIV)** – Perform citation anomaly detection, semantic verification, and AI-authorship analysis.
4. **Database (PostgreSQL)** – Stores user details, manuscript metadata, reports, and analysis results.

This layered approach separates concerns and ensures that individual components can be updated independently.

a. Overview of Web Service / Mobile Service

The **web service** acts as the primary interaction point for users:

- **Author Features:** Upload manuscripts (PDF/DOI), receive feedback, and access reports.
- **Editor Features:** View analysis reports, visualize citation graphs, and assess flagged content with integrity scores.

- **Extensibility:** While initially designed as a web platform, the architecture supports future extension into mobile applications for broader accessibility.

The web service communicates with the backend APIs to ensure smooth integration between users, AI modules, and the database.

b. Overview of AI/DS Module Architecture

The AI/DS architecture integrates two modules:

- **CSAD:** Builds and analyzes citation graphs, flags anomalies such as excessive self-citations and reciprocal citation patterns.
- **SAIV:** Uses semantic embeddings (SciBERT) for text alignment checks and machine learning classifiers to detect AI-generated content.

Both modules are exposed via API endpoints, making them modular and independently upgradable without affecting the overall system.

i. Summary of Method

The working methodology involves the following steps:

1. Author uploads manuscript (PDF/DOI).
2. Manuscript text and references are extracted and preprocessed.
3. CSAD constructs a citation graph and detects anomalies.
4. SAIV checks semantic consistency and detects AI-generated content.
5. Results are consolidated into a report with visualizations and risk scores.
6. Editors access the report via a secure dashboard.

This hybrid methodology ensures multi-layered verification of manuscripts, covering citations, semantics, and authorship authenticity.

c. Overview of API

The backend APIs are implemented using **FastAPI** and serve as the communication

layer between the frontend, AI/DS modules, and the database.

- **/upload_manuscript** – Accepts file/DOI input and stores metadata.
- **/analyze_citations** – Triggers CSAD for citation graph generation and anomaly detection.
- **/analyze_authorship** – Invokes SAIV for semantic integrity and AI-authorship verification.
- **/get_report** – Fetches consolidated integrity reports for editor review.

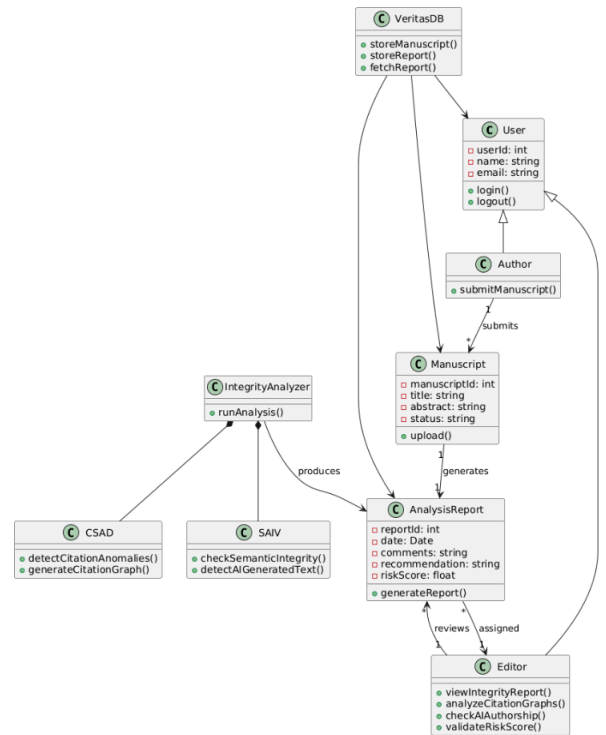
All APIs follow REST architecture and are documented with **Swagger/OpenAPI** for easy integration and testing.

3. Class Diagram

The class diagram illustrates the structural design of the VERITAS platform and the interactions between its main components.

1. **User Roles** – The system defines a generic User class, extended by Author and Editor. Authors can submit manuscripts, while Editors review integrity reports, analyze citation graphs, check AI authorship, and validate the overall risk score.
2. **Core Entities** – Manuscripts uploaded by Authors are stored in the system with details such as title, abstract, and status. Each manuscript is linked to an AnalysisReport, which records the findings, comments, recommendations, and a calculated risk score.
3. **AI/DS Modules** – The IntegrityAnalyzer manages automated checks with two submodules: CSAD (for citation anomaly detection) and SAIV (for semantic integrity and AI authorship verification). These modules generate integrity results that are compiled into reports.
4. **Database Layer** – The VeritasDB handles persistent storage of manuscripts, reports, and user information, ensuring data can be retrieved and updated efficiently.

This structure ensures smooth interaction between users, the analysis modules, and the database, providing a clear workflow from manuscript submission to final editor validation.



4. Sequence Diagram

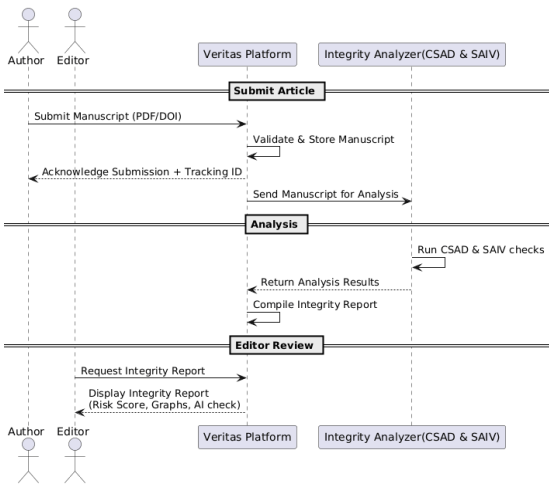
The sequence diagram illustrates the end-to-end process of handling a manuscript in the VERITAS platform.

1. **Submission** – The Author uploads a manuscript (PDF or DOI). The Veritas Platform validates and stores it, provides a tracking ID, and forwards the paper for analysis.
2. **Analysis** – The Integrity Analyzer runs two checks: CSAD to detect citation anomalies, and SAIV to verify AI authorship. The results are sent back and compiled into an Integrity

Report.

3. **Review** – The Editor requests the Integrity Report. The Veritas Platform displays the report with risk score, citation graphs, and AI authorship findings, allowing the Editor to evaluate the manuscript’s integrity.

This workflow ensures that manuscript checks are automated and consistent, while the final validation remains under the Editor’s control.



5. Activity Diagrams

The activity diagram for the **VERITAS platform** illustrates the workflow of manuscript submission, integrity verification, and editorial decision-making. It captures the interaction between authors, the manuscript processing system, AI/DS modules, and editors.

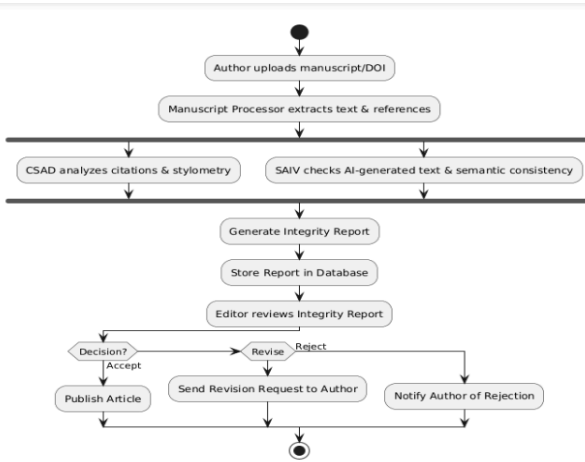
The process begins when the **Author uploads a manuscript or DOI**. The **Manuscript Processor** extracts the text and references, which are then analyzed by two AI/DS modules:

- **CSAD (Citation & Stylometry Anomaly Detector)**, which checks for citation irregularities and stylometric anomalies.
- **SAIV (Semantic Integrity & AI Verifier)**, which detects AI-generated content and evaluates semantic consistency across manuscript sections.

Following the analysis, the system generates an **Integrity Report**, which is stored in the database and made accessible to the **Editor**. The editor reviews the report and makes a decision:

- **Accept** – The manuscript proceeds to publication.
- **Revise** – A revision request is sent back to the author.
- **Reject** – The author is notified of rejection.

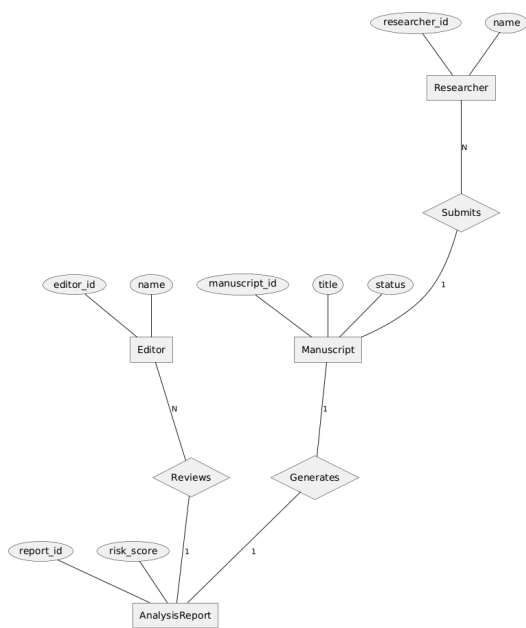
This activity diagram clearly shows how different modules and users interact to ensure manuscript integrity and streamline the publication process.



6. Database Design

The ER diagram illustrates the key entities and relationships in the VERITAS platform. A **Researcher** entity stores details such as researcher ID and name. Researchers can submit one or more **Manuscripts**, each identified by attributes like manuscript ID, title, and status. Every manuscript undergoes analysis and **generates** a single **AnalysisReport**, which contains attributes such as report ID and risk score. Each report is then **reviewed** by an **Editor**, who is represented by attributes like editor ID and name.

This structure clearly models how manuscripts flow through the system — from submission by researchers, to automated analysis, and finally review by editors — forming the foundation of the VERITAS database design.



7. User Interface Design

The user interface for the VERITAS platform is designed for both researchers and editors, providing a clear and secure workflow. The process begins with user login and authentication, which directs users to a personalized dashboard.

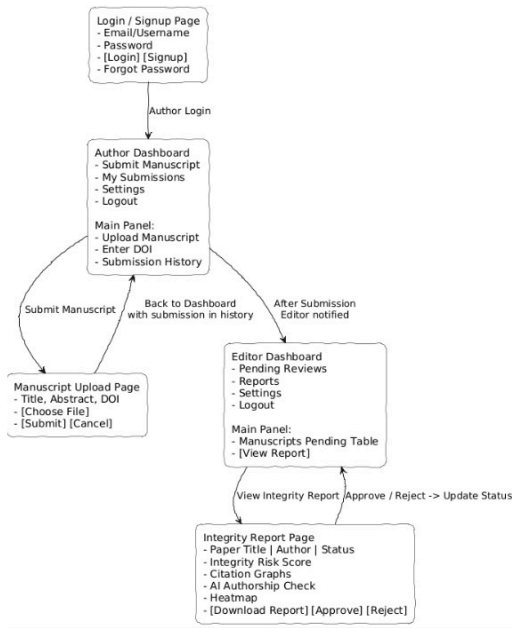
From the dashboard, a researcher can upload a document for analysis. The system uses a suite of AI modules to perform a detailed integrity analysis. The core of this analysis is handled by the Integrity Analyzer, which includes:

- **Check AI Authorship (SAIV)**: This module determines if the text was written by an AI.
- **Analyze Citation Graphs (CSAD)**: This module identifies anomalous citation patterns.

After the analysis, the system generates a detailed report, which is accessible from the user’s dashboard. While researchers can view the analysis history, only the editor has the final write access to approve or publish a research paper after reviewing the report. This ensures a secure and controlled publishing workflow.

The entire platform is built with robust security measures like HTTPS and secure API keys to protect data and is designed to act as an intelligent decision-support tool.

UI Mockups / Wireframes with Navigation Flow



8. Tools and Technologies

The VERITAS platform is developed using a modular microservices-based architecture. The backend is implemented with **FastAPI**, while the frontend uses **React.js** combined with **D3.js** for interactive visualizations. The database layer is powered by **PostgreSQL** to ensure reliability and scalability. AI and data science modules, including **CSAD** and **SAIV**, are built using **PyTorch**, **Hugging Face Transformers**, and **NetworkX** for citation analysis. Deployment is handled through **Docker** and **Kubernetes**, ensuring portability and efficient orchestration. The entire system is version-controlled using **GitHub** with GitFlow strategy, and testing is carried out with tools such as **Pytest** and **Cypress**.

a) Hardware Requirements

The platform is designed for cloud deployment, which minimizes end-user hardware requirements. Authors and editors only require a standard device with internet access and a modern web browser. For development and testing, the system requires at least:

- Processor: Intel Core i5 or higher
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 256 GB SSD
- GPU: NVIDIA GPU with CUDA support (for model training and testing)

Cloud infrastructure such as **Google Cloud Platform (GCP)** or **Amazon Web Services (AWS)** will be used for production, ensuring scalability and high availability.

b) Software Requirements

The software stack for VERITAS includes:

- **Operating System:** Windows 10/11 or Linux (Ubuntu preferred for deployment)
- **Backend:** FastAPI (Python-based REST framework)
- **Frontend:** React.js with D3.js for visualization
- **Database:** PostgreSQL (Cloud SQL / AWS RDS)
- **AI/ML Libraries:** PyTorch, Hugging Face, SciBERT, NetworkX
- **Containerization:** Docker, Kubernetes for orchestration
- **Version Control:** GitHub with GitFlow branching strategy
- **Testing Tools:** Pytest (backend), Cypress (frontend), Locust (load testing)
- **Deployment:** GCP (Cloud Run, Kubernetes Engine) or AWS (Fargate, EKS)

<<Veritas: An AI-Powered Platform for Proactive Research Integrity Analysis>>

TESTING

1. Unit Testing

Unit testing for the Veritas platform will focus on verifying the core software components in isolation. This is done to ensure each function and module behaves as expected before they are integrated with other parts of the system. The project proposal and SRS documents outline several key software functionalities that will be tested.

A summary of the unit testing to be conducted is as follows:

- **User Management:** This includes testing core functionalities such as account creation, user login, and session handling to ensure they are secure and work as intended.
- **Document Processing:** Tests will verify the functions responsible for uploading manuscripts, extracting metadata, text, and references from PDFs or DOI links.
- **Report Generation:** This will involve testing the components that compile data from the AI/DS modules to generate easy-to-understand reports with visual elements like heatmaps and citation graphs.
- **Data Validation:** Unit tests will ensure that all data inputs, such as file formats and DOI links, are properly validated to prevent system errors.
- **Database Interactions:** The functions for storing and retrieving data from the **PostgreSQL** database will be tested to confirm reliable data management.
- **Backend-Frontend Communication:** The API calls that handle the communication between the frontend and backend will be tested to ensure secure and efficient data exchange.

The testing will be conducted using frameworks like

Pytest for backend logic and **Jest** for frontend components.

a. Sample Test Cases

#	TS1					
Title						
User Authentication and Manuscript Submission Functionality						
Description						
Tests user login/logout operations, account creation, authentication validation, PDF manuscript uploads, and DOI-based manuscript submission to verify individual component reliability.						
#	Summary	Depend ency	Pre-condition	Post-condition	Execution Steps	Expected Output
TC1	Verify that a new user can successfully create an account and log in to the dashboard.		New user with no existing account	The user account is created, and the user is authenticated.	1. Navigate to the Veritas platform. 2. Select the account creation option. 3. Enter a valid email and password. 4. Log in with the new credentials.	The user is granted access to a secure dashboard for authors or editors.
TC2	Verify that an existing user can log in with correct credentials.		User has a pre-existing account	Users are authenticated and redirected to their dashboard.	1. Navigate to the Veritas platform. 2. Enter the registered email and password. 3. Click "Login."	The user is redirected to the user dashboard where they can upload manuscripts or view reports.
TC3	Verify that a user cannot log in with an incorrect password.		No registered account for the user.	The login attempt is denied.	1. Navigate to the Veritas platform. 2. Enter an email and password for a non-existent account. 3. Click "Login."	The system displays a "Login failed" error message.

2. AI Module Testing

The AI/DS modules will be tested using standard metrics and methodologies to ensure their accuracy and robustness. This process is crucial because these modules form the analytical core of the Veritas platform. The testing will be conducted with the following focus areas:

- Model Evaluation: The performance of the AI models will be measured using standard metrics such as F1-score, precision, and recall to ensure they are accurate in their predictions.
- Validation Dataset: The models will be tested on a combination of real and synthetic content. This is particularly important for the SAIV module, as it needs to be effective at identifying AI-generated text.
- Adversarial Testing: The models' robustness will be checked by inputting paraphrased or translated text to ensure they cannot be easily deceived.
- Interpretability Checks: The system will be designed to provide clear, explainable results so that the reasons for flagging an issue are understandable to the user.

a.Sample Test Cases

#	TS1					
Title	CSAD and SAIV Module Accuracy and Detection					
	Tests AI-generated text detection, human-authored text verification, semantic consistency analysis, citation anomaly detection, retracted reference identification, and integrated module performance with confidence scoring.					
Summary	Dependency	Pre-condition	Post-condition	Execution Steps	Expected Output	

TC1	Verify that a manuscript containing known AI-generated text is flagged	The SAIV module has been trained on AI-generated text datasets	SAIV module is active and manuscript text is available	AI authorship detection provides confidence scores for flagged sections	1. Submit manuscript with AI-generated content 2. SAIV scans using trained algorithms 3. Review confidence scores and flagged sections	System highlights AI-generated sections with high probability scores and specific passage identification
TC2	Verify that human-authored manuscripts are not falsely flagged as AI-generated	SAIV module is operational and calibrated	Human-authored test manuscript is available	System provides low AI-likelihood scores	1. Submit human-written manuscript 2. SAIV performs authorship analysis 3. Analyze confidence scores	System returns low AI-likelihood scores without flagging sections as suspicious
TC3	Verify detection of semantic inconsistencies between manuscript sections	SAIV module with SciBERT embeddings is operational	Test manuscript with semantic misalignment is available	Semantic inconsistencies are detected with confidence scores	1. Submit manuscript with misaligned sections 2. SAIV performs semantic analysis 3. Calculate cosine similarity scores 4. Review consistency report	System flags semantic inconsistencies with low similarity scores and explanatory feedback

TC4	Verify detection of suspicious citation patterns and self-citation abuse	CSAD module with NeworkX is operational	Test manuscript with citation anomalies is available	Citation anomalies are detected and visualized	1. Submit manuscript with citation irregularities 2. CSAD builds citation network graph 3. Analyze citation patterns 4. Review anomaly report	System generates citation graphs highlighting suspicious patterns with anomaly scores
TC5	Verify identification of citations to retracted papers	CSAD has access to retracted paper databases	Test manuscript citing retracted papers is available	Retracted references are flagged in report	1. Submit manuscript with retracted citations 2. CSAD cross-references against databases 3. Identify flagged references 4. Review citation integrity report	System identifies retracted references with clear warnings and review recommendations
TC6	Verify combined CSAD and SAIV modules produce consolidated integrity reports	Both modules are operational and integrated	Test manuscript requiring full analysis is available	Consolidated report with unified risk scoring	1. Submit manuscript for complete analysis 2. Both modules process simultaneously 3. System consolidates results 4. Review integrated report	Comprehensive integrity report with citation analysis, AI detection, semantic checks, and unified risk score

3. Integration Testing

Integration testing for the Veritas platform is designed to validate the seamless communication and data flow between all system components. Given the microservices architecture, this type of testing is critical to ensuring the system's overall reliability and robustness. The tests will specifically verify the interactions between the frontend, the FastAPI backend, and the containerized AI/DS modules.

The testing process will include:

- Module Communication Testing: This will validate REST API calls between the backend and the AI/DS modules.
- Data Flow Testing: This ensures the correct JSON data structure and end-to-end logic are maintained as information moves through the system.
- Load Testing: Using tools like Locust, the system's performance will be assessed under simulated high traffic to ensure it can handle multiple user uploads and API calls without degradation.

a.Sample Test Cases

#	TS1					
Title	Integration Testing Title: API Communication and Cross-Component Data Flow					
Description	Tests manuscript submission to report generation workflow, editor report access with visualizations, and retracted reference handling to verify seamless integration between frontend, backend APIs, and AI/DS modules.					
	Summary	Dependency	Pre-condition	Post-condition	Execution Steps	Expected Output
TC1	Manuscript Submission and Report Generation	Frontend (React), Backend (FastAPI), AI/DS Modules (CSAD & SAIV), Database (PostgreSQL)	User is logged in. A manuscript file (e.g., PDF) is ready for upload.	A new entry exists in the database, and an analysis report is generated and accessible to the user.	1. Log in as an Author. 2. Navigate to the "Submit Manuscript" page. 3. Upload a PDF document. 4. Wait for the system to process the document. 5. Navigate to the "My Reports" section.	The uploaded manuscript appears in the "My Reports" list with a generated Integrity Risk Score and a link to the detailed report.

TC2	Editor Accessing AI Analysis	Frontend (React), Backend (FastAPI), AI/DS Modules (CSAD & SAIV), Database (PostgreSQL)	A manuscript has been submitted and fully processed, and an analysis report has been generated. The user is logged in as an Editor.	The Editor can view all visual and textual elements of the report.	1. Log in as an Editor. 2. Navigate to the list of processed manuscripts. 3. Select a processed manuscript to view its report. 4. Verify that the Integrity Risk Score is displayed. 5. Click on the Citation Graph and Semantic Heatmap to ensure they load and display correctly.	The report page loads successfully, showing the Integrity Risk Score, a clickable citation graph with accurate nodes/edges, and a text heatmap highlighting areas of concern.
TC3	Handling of Retracted References	Frontend (React), Backend (FastAPI), CSAD Module, Database (PostgreSQL)	A manuscript containing a known retracted reference is uploaded and processed.	The report correctly flags the retracted reference.	1. Upload a test manuscript containing a citation to a known retracted paper. 2. Wait for the system to process the document. 3. As an Editor, access the generated report. 4. Navigate to the citation analysis section.	The system's report highlights the specific reference and indicates that it has been retracted.

4. System Testing

System testing for the Veritas platform will validate the entire system's functionality in a complete, integrated environment. It focuses on verifying that all components—including the frontend, backend, AI/DS modules, and database—work together seamlessly to meet the project's functional and non-functional requirements. This stage of testing is crucial for identifying issues that may not be apparent during unit or integration testing, such as performance bottlenecks or data consistency problems.

The system testing process will include:

- End-to-End Functional Testing:** This involves verifying the complete user workflow, from a researcher uploading a manuscript to an editor viewing the final integrity report. This ensures that every feature, as outlined in the proposal, works correctly.

- Performance Testing:** This will measure the system's responsiveness and stability under various loads, ensuring it can handle simultaneous users and large document uploads.
- Security Testing:** This is a critical part of system testing that checks for vulnerabilities in user authentication, session management, and data protection.
- Usability Testing:** This will evaluate the user interface and overall user experience to ensure the platform is intuitive and easy for both authors and editors to navigate.
- Regression Testing:** This will be performed after any major changes to the system to ensure that new code has not negatively impacted existing functionality.

a.Sample Test Cases

#	TS1					
Title	System Testing Title: End-to-End System Validation and Performance					
Description	Tests complete user workflows from author manuscript submission to editor report viewing, role-based access control and security authentication, and system performance under concurrent load to validate overall platform functionality.					
#	Summary	Depend-ency	Pre-condition	Post-condition	Execution Steps	Expected Output
TC1	Overall System Functionality	All system components (Frontend, Backend, Database, AI Module s)	The system is fully deployed and accessible. A user account (both Author and Editor) exists.	The complete workflow from document upload to report viewing is successful.	1. Log in as an Author. 2. Upload a manuscript and wait for the analysis to complete. 3. Log out and log in as an Editor. 4. Access and view the generated integrity report.	The report is loaded and displayed correctly for the Editor, and all functionalities are accessible without errors.

TC2	Security and User Authentication	Session Management Module, User Authentication	The system is running. An Author and an Editor account exist with different permissions.	Access control is correctly enforced, preventing unauthorized actions based on user roles.	1. Log in as an Author. 2. Attempt to access a page or perform an action only allowed for an Editor (e.g., approving a paper). 3. Log out and try to access a page without credentials.	The system denies the Author's attempt to perform an Editor-only action and redirects unauthenticated users to the login page.
TC3	Performance Under Load	Backend, Database, AI Modules	A test environment is set up with load testing tools (e.g., Locust).	The system remains stable and responsive under a simulated load of multiple concurrent users.	1. Simulate 100 concurrent users logging in. 2. Simulate 50 concurrent manuscript uploads and analysis requests. 3. Monitor system metrics (response time, CPU usage, etc.).	The system's response time remains within acceptable limits, and no system crashes or unexpected errors occur.

Appendix C: Project Proposal and Design Presentation Slides

VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis

PROJECT ABSTRACT & PROPOSAL PRESENTATION

Name of the Students

- Billvin Christo Thomas
- Hrishikesh S
- K V Chandana Chithran
- Liz Maria Benny

Name of the Guide :

- Dr.Varghese S.Chooralil



Dept. of AIDS, RSET (Autonomous), Kochi

1

Introduction

Introduction

- **Problem:** The number of retracted scientific papers is increasing, eroding public trust in science and wasting research funds. Existing tools primarily check for plagiarism, leaving critical gaps.
- **Our Solution:** We propose Veritas, a web-based platform that uses a suite of AI modules to perform a deep, multi-faceted integrity analysis of research papers before publication.
- **Core Idea:** To act as an intelligent "decision-support" tool for editors and researchers, flagging statistical anomalies and potential integrity issues that are difficult for humans to spot manually.
- **Social Relevance:** By promoting research integrity, Veritas helps ensure the reliability of the scientific record, which is crucial for societal progress and public health.



Dept. of AIDS, RSET (Autonomous), Kochi

2

Background

Background

- Academic retractions are rising, not just for plagiarism but due to subtler issues like AI-generated text and citation manipulation. Researchers now analyze citation networks to spot patterns like self-citation clusters or mutual citation rings.
- Meanwhile, generative AI tools are becoming sophisticated enough to imitate academic writing, making detection trickier.
- To tackle this, explainable machine learning models offer transparency, showing why a manuscript might be flagged.
- Platforms like Veritas bring these innovations together, helping detect integrity risks early and support fair publishing.



Dept. of AIDS, RSET (Autonomous), Kochi

3

Project Objectives

Project Objectives

List of Software functionalities

- Secure** **User** **Dashboard**
For account creation, file/DOI upload, and viewing analysis history
- Interactive** **Reports**
Visual display of **Integrity Risk Score**, citation graphs, and text heatmaps
- Automated** **Document** **Processing**
Extracts metadata, text sections, and references from uploaded documents
- Modular** **Backend**
Built with REST APIs to communicate with AI modules for scalability
- Journal** **Integration** **Ready**
Extensible API for future integration with editorial systems



Dept. of AIDS, RSET (Autonomous), Kochi

4

Project Objectives

List of AI/DS Modules

1.CSAD – Citation & Stylometry Anomaly Detector

- Detects excessive self-citations, citation rings, etc.
- Flags references to retracted papers

2.SAIV – Semantic Integrity & AI Authorship Verifier

- Checks for semantic mismatches
- Detects AI-generated text (e.g., GPT-style)



Literature Survey

Literature Survey

Title of Paper, Authors	Journal / Conference	Year	Technologies / Algorithms Used	Advantages / Features	Limitations
Deep graph learning for anomalous citation detection	IEEE TNNLS	June 2022	GLAD framework integrating Doc2Vec (text features), DGI + GCN (node features), Deep Autoencoder (edge features), SciBERT + SVM (citation purpose), and Logistic Regression (anomaly detection).	Combines semantic, relational, and structural cues; high precision; works with partial labels; integrates node + edge features	Requires labeled citation purposes for training; annotation and dataset creation are time-consuming; computationally intensive for large-scale citation networks
Accurately detecting AI text when Chat GPT is told to write like a chemist	Cell Rep. Phys. Sci.	2023	XGBoost on 20 text features (no perplexity)	98–100% accurate; works on GPT-3.5 & 4; resists prompt tricks	Only for chemistry-style writing; may fail on other domains
Anomalous citations detection in academic networks	AI Review	2024	ACTION: Doc2Vec, NMF, author/journal modeling, CPU, semi-supervised learning	Uses text + author/journal relations; handles heterogeneous networks	Needs rich metadata; heavy for big datasets



Literature Survey

Literature Survey

Title of Paper, Authors	Journal / Conference	Year	Technologies / Algorithms Used	Advantages / Features	Limitations
Testing of detection tools for AI-generated text	International Journal for Educational Integrity	2023	Evaluation of 14 AI-text detectors (e.g., Turnitin, GPTZero)	Broad tool coverage; realistic test scenarios; examines bias & obfuscation effects	Low accuracy for AI detection; bias toward "human-written"; poor with paraphrased/translated text
Bibliometric feature identification and analysis of retracted papers in biomedicine: An interpretable machine learning perspective	Information Processing & Management	2025	XGBoost; SHAP; bibliometric feature analysis	High accuracy (87%); interpretable ML; generalizable to other domains	Depends on WoS data; may not fit non-biomedicine without retraining



Product Survey

Product Survey

Product	Pros	Cons	Market Position
Turnitin Similarity Check	Established market presence with over 40 million users. Large reference database of over 70 billion pages. Integration with more than 15,000 institutions. Provides real-time feedback for students.	Limited to text similarity only. Lacks citation pattern analysis. No AI content detection capabilities. High subscription cost at \$3-6 per student. 30-40% false positive rate.	A dominant force in the plagiarism detection market, the company was acquired for \$1.75 billion in 2019.
CrossRef Similarity Check (iThenticate)	Focuses on scholarly content with over 60 million articles. Integrated with over 700 publishers. Allows for pre-publication checking. Provides detailed similarity reports.	Expensive enterprise model, costing \$100-200 per user annually. Lacks semantic or citation analysis. Limited AI detection capabilities. Features a complex user interface.	Considered the industry standard for publishers
Grammarly Business	Offers real-time writing assistance. Boasts over 30 million daily active users. Includes style and tone analysis. User-friendly interface.	Not specifically designed for academic integrity. Lacks citation or reference analysis. Limited to grammar and style suggestions. Does not detect research misconduct	A leader in the writing assistance market with a \$13 billion valuation



Product Survey

Product	Pros	Cons	Market Position
CopyLeaks AI Content Detector	Detects AI-generated content. Supports over 100 languages. API is available for integration. Offers real-time detection.	Accuracy is limited to 65-75% for academic text. Lacks understanding of academic context. No features for checking citation integrity. Reporting is basic and lacks explanation. High false positive rate for technical writing.	A growing player in the AI detection market.
Winston AI	Specializes in AI content detection. Claims a 99.6% accuracy rate. Supports multiple AI models. Includes a plagiarism detection combo.	Limited to only AI detection. Lacks academic integrity features. No citation analysis capabilities. Expensive pricing at \$12-20 per month. Not validated on scientific papers.	An emerging tool in the AI detection field.



Methodology for Software Services

- **Frontend:** React + D3.js for interactivity
- **Backend:** FastAPI, PostgreSQL
- Modular REST-based architecture
- GitHub-based version control with GitFlow



Methodology for AI/DS Services

- CSAD: NetworkX for citation pattern analysis
- SAIV: Sci-BERT + custom classifiers for AI content
- Gemini API to generate fake AI data for training
- Model integration via REST APIs



Development of Software Services

- Phase 1: Account management, document upload
- Phase 2: Report generation
- Phase 3: API development



Development of AI/DS Modules

- CSAD: Citation graph + stylometric fingerprinting
- SAIV: Semantic section alignment + authorship scoring
- Built using PyTorch and Hugging Face



Integration

- Architectural Pattern: Microservices
- Communication Protocol: The main FastAPI backend will communicate with the Dockerized AI modules via a REST API.
- Data Format: JSON will be used for all data exchange between services.
- Orchestration: The final deployment will use Kubernetes to manage and scale the containerized AI modules independently.



Software functionalities

- **Unit Testing**
Test core components like login, upload, and dashboard using Pytest.
- **API Testing**
Verify frontend-backend communication via REST APIs.
- **UI Testing**
Use Cypress to simulate user actions and ensure smooth interactions.
- **Functional Testing**
Check features like upload, report view, and history tracking.
- **Security Testing**
Validate authentication and session handling for data protection.



AI/DS Modules

- **Model Evaluation**
Use F1-score, precision, and recall to measure accuracy.
- **Validation Dataset**
Test on real and synthetic (AI-generated) content.
- **Adversarial Testing**
Input paraphrased or translated text to check robustness.
- **Interpretability Checks**
Ensure explainability of flagged issues in the output.



Integration

- Module**
 Validate REST API calls between backend and AI modules.
- Data**
 Ensure correct JSON data structure and end-to-end logic.
- Load**
 Use Locust to simulate multiple uploads and assess system performance.
- Deployment**
 Test Docker containers and microservice orchestration on Kubernetes.



Dept. of AIDS, RSET (Autonomous), Kochi

17

Deployment Plan

- Cloud Platform:**
 - Deploy on **Google Cloud Platform (GCP)** or **Amazon Web Services (AWS)**
- Frontend Deployment:**
 - React.js app hosted using **GCP Cloud Storage** or **AWS Amplify**
- Backend Deployment:**
 - FastAPI backend packaged in Docker and deployed via **GCP Cloud Run** or **AWS Fargate**
- Database:**
 - Use **PostgreSQL** via **Cloud SQL (GCP)** or **RDS (AWS)**
- AI/DS Modules (CSAD & SAIV):**
 - Deployed as **Docker-based microservices** using **Kubernetes (GKE or EKS)**
- CI/CD:**
 - Automated using **GitHub Actions** for testing, building, and deploying
- Security & Monitoring:**
 - Enforce **HTTPS**, secure API keys using **Secret Manager**, and monitor with **Cloud-native tools**
- Domain & Access:**
 - Custom domain with **SSL certificate**, accessible via **Load Balancer**



Dept. of AIDS, RSET (Autonomous), Kochi

18

Documentation and Version Control

Plan for Documentation and Version Control

- Version Control:** A private GitHub repository will be used as the single source of truth for all code, managed with the GitFlow branching strategy.
- Code Documentation:** We will use inline comments and clear docstrings for all functions.
- API Documentation:** The FastAPI backend will automatically generate interactive API documentation using OpenAPI (Swagger).
- Project Page:** A simple public-facing website will be created using GitHub Pages to showcase the project's goals and progress.



Dept. of AIDS, RSET (Autonomous), Kochi

19

Timeline

Timeline

- Weeks 1–4: Finalize project theme, domain, and scope
- Weeks 5–6: Submit and present proposal
- Weeks 7–9: Conduct requirement analysis and literature survey
- Weeks 10–12: Design core architecture and submit design report
- Weeks 13–14: Review and improve code from first development cycle



Dept. of AIDS, RSET (Autonomous), Kochi

20

Conclusion

- Veritas tackles multi-layered academic integrity challenges.
- Blends citation forensics, NLP, and AI authorship analysis.
- A unique and timely solution for maintaining trust in academic publishing.



References

References

- [1] J. Liu, F. Xia, X. Feng, J. Ren, and H. Liu, "Deep graph learning for anomalous citation detection," IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 6, pp. 2543–2557, Jun. 2022, doi: 10.1109/TNNLS.2022.3145092.
- [2] H. Desaire, A. E. Chua, M.-G. Kim, and D. Hua, "Accurately detecting AI text when Chat GPT is told to write like a chemist," Cell Reports Physical Science, vol. 4, Nov. 2023, Art. no. 101672, doi: 10.1016/j.xcrp.2023.101672.
- [3] J. Liu, X. Bai, M. Wang, S. Tuarob, and F. Xia, "Anomalous citations detection in academic networks," Artificial Intelligence Review, vol. 57, Art. no. 103, Mar. 2024, doi: 10.1007/s10462-023-10655-5.
- [4] D. Weber-Wulff, A. Anohina-Naumeca, S. Bjelobaba, T. Foltýnek, J. Guerrero-Dib, O. Popoola, P. ˇ Sigut, and L. Waddington, "Testing of detection tools for AI-generated text," International Journal for Educational Integrity, vol. 19, Art. no. 26, 2023, doi: 10.1007/s40979-023-00146-z.
- [5] J. Liu, X. Wang, and X. Liang, "Bibliometric feature identification and analysis of retracted papers in biomedicine: An interpretable machine learning perspective," Information Processing & Management, vol. 62, 2025, Art. no. 104176, doi: 10.1016/j.ipm.2025.104176.



*Thank
you*



VERITAS: An AI-Powered Platform for Proactive Research Integrity Analysis

PROJECT DESIGN PRESENTATION & INTERMEDIATE CODE REVIEW

Name of the Students

- Billvin Christo Thomas
- Hrishikesh S
- K V Chandana Chithran
- Liz Maria Benny

Name of the Guide :

- Dr.Varghese S.Chooralil



Dept. of AIDS, RSET (Autonomous), Kochi

1

Project Objectives

Project Objectives -

List of Software functionalities

1. **Secure User Dashboard**
 - Account creation & login for authors/editors.
 - Upload manuscript via **PDF**.
 - View analysis history and reports.
2. **Automated Document Processing**
 - Extracts **metadata, sections, and references** from uploaded papers.
 - Preprocesses content for AI/DS modules.
3. **Interactive Integrity Reports**
 - Displays **Integrity Risk Score**, citation graphs, and heatmaps.
 - Shows flagged AI-generated or inconsistent text.



Dept. of AIDS, RSET (Autonomous), Kochi

2

Project Objectives

Project Objectives -

List of Software functionalities

4. **Modular Backend (FastAPI)**
 - Handles communication between AI modules and frontend.
 - Scalable via REST APIs.
5. **Journal Integration Ready**
 - API support for integration with **editorial or journal systems**.
6. **Role-Based Access Control**
 - Authors can view reports; editors can review, approve, or reject papers.



Dept. of AIDS, RSET (Autonomous), Kochi

3

Project Objectives

Project Objectives -

List of AI/DS Modules

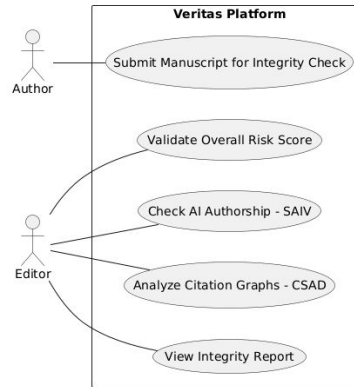
1. **CSAD** – Citation & Stylometry Anomaly Detector
 - Detects excessive self-citations, citation rings, etc.
 - Flags references to retracted papers
2. **SAIV** – Semantic Integrity & AI Authorship Verifier
 - Checks for semantic mismatches
 - Detects AI-generated text (e.g., GPT-style)



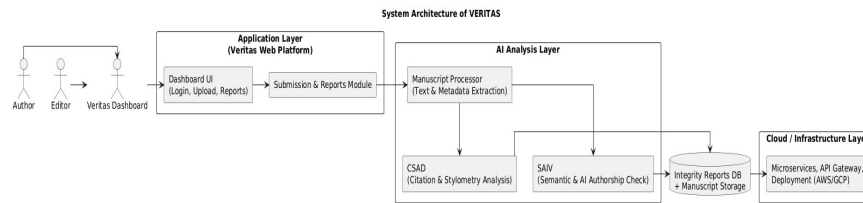
Dept. of AIDS, RSET (Autonomous), Kochi

4

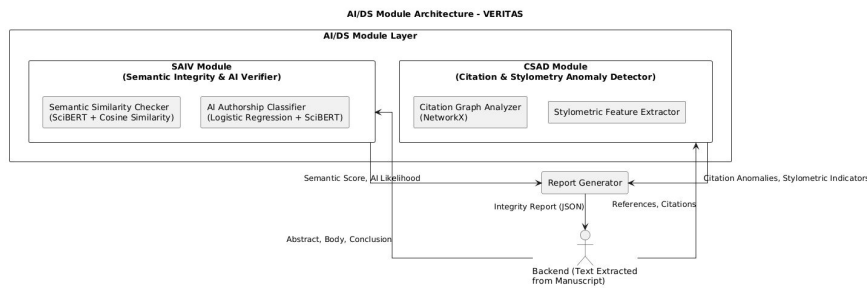
Use Case Diagram



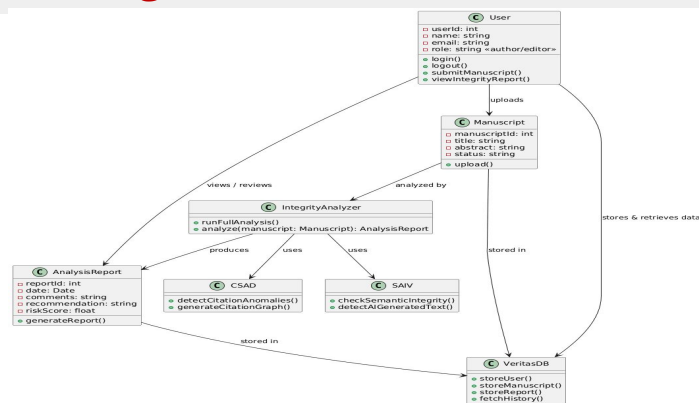
System Architecture



AI/DS Module Architecture



Class Diagram



Summary of Data Sources for AI/DS Module

1. CSAD (Citation & Stylometry Anomaly Detector)

Data Sources:

- Uploaded research paper (PDF) — extracted using *PyPDF2*
- References parsed via *Regex*
- Retraction database (*Retraction Watch CSV*)
- Citation network built with *NetworkX*

Use:

- Detects **self-citations**, **retracted references**, and **citation rings/cartel**
- Analyzes **citation density** to flag abnormal patterns and writing-style anomalies.

Output:

- **Integrity score** (0–1) indicating citation ethics
- **Interactive citation graph** showing author relationships



Summary of Data Sources for AI/DS Module

2. SAIV (Semantic Integrity & AI Verifier)

Data Sources:

- Uploaded manuscript text (Abstract, Body, and Conclusion extracted using *PyMuPDF* + *Regex*)
- **AI vs Human Text Classification Dataset (Kaggle)** — contains both AI-generated and human-written samples

Use:

- Checks **semantic consistency** between Abstract and Conclusion (using **SciBERT + cosine similarity**)
- Detects **AI-generated content** using a **Logistic Regression classifier** trained on Kaggle data

Output:

- Semantic similarity score
- AI-likelihood probability and flag



References

References

- [1] J. Liu, F. Xia, X. Feng, J. Ren, and H. Liu, "Deep graph learning for anomalous citation detection," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 6, pp. 2543–2557, Jun. 2022, doi: 10.1109/TNNLS.2022.3145092.
- [2] H. Desaire, A. E. Chua, M.-G. Kim, and D. Hua, "Accurately detecting AI text when Chat GPT is told to write like a chemist," *Cell Reports Physical Science*, vol. 4, Nov. 2023, Art. no. 101672, doi: 10.1016/j.xcrp.2023.101672.
- [3] J. Liu, X. Bai, M. Wang, S. Tuarob, and F. Xia, "Anomalous citations detection in academic networks," *Artificial Intelligence Review*, vol. 57, Art. no. 103, Mar. 2024, doi: 10.1007/s10462-023-10655-5.
- [4] D. Weber-Wulff, A. Anohina-Naumeca, S. Bjelobaba, T. Folt'ynek, J. Guerrero-Dib, O. Popoola, P. ˇ Sigut, and L. Waddington, "Testing of detection tools for AI-generated text," *International Journal for Educational Integrity*, vol. 19, Art. no. 26, 2023, doi: 10.1007/s40979-023-00146-z.
- [5] J. Liu, X. Wang, and X. Liang, "Bibliometric feature identification and analysis of retracted papers in biomedicine: An interpretable machine learning perspective," *Information Processing & Management*, vol. 62, 2025, Art. no. 104176, doi: 10.1016/j.ipm.2025.104176.



Thank
you



**Appendix D:
Vision, Mission, Programme
Outcomes, Course Outcomes and
CO-PO-PSO Mapping**

Rajagiri School of Engineering & Technology (Autonomous)

Vision & Mission

Vision: To evolve into a premier technological and research institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Mission: To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Objective: The institution has the broad objective of being an active agent of change by responding to the needs and challenges of the times. This is achieved basically through the process of education, training and research.

Department of Artificial Intelligence and Data Science

Vision : To become a centre of excellence in Artificial Intelligence and Data Science and moulding professionals by imparting high-quality knowledge for the sustainable betterment of mankind.

Mission : To empower individuals with expertise in constantly evolving AI and Data Science tools and technologies for impactful innovations, research and societal advancements through ethical practices and industrial collaborations.

PROGRAMME EDUCATIONAL OBJECTIVES

PEO1: Graduates will thrive in their careers by mastering both foundational and advanced technical skills in Artificial Intelligence and Data Science, enabling them to adapt to and excel with emerging technology.

PEO2: Graduates will foster a culture of continuous learning in Artificial Intelligence and Data Science, ensuring their professional relevance and adaptability in an ever-evolving industry.

PEO3: Graduates will demonstrate strong teamwork, and effective communication skills while developing responsible and sustainable AI solutions ethically with societal impact.

Component	Purpose
PO (Programme Outcome)	Broad graduate attributes set by NBA
WK (Knowledge/Attitude Profile)	Type and level of knowledge required (NBA-defined)
CO (Course Outcome)	What students should be able to do after the course

PI (Performance Indicator)	Specific measurable skills under each PO
-----------------------------------	--

Program Outcomes (PO)

(As per National Board of Accreditation (NBA))

PO1: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profile (WK)

A Washington Accord program provides:

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences
WK2: Conceptually-based mathematics , numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline
WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline
WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area
WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline
WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development*
WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues
WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes

Programme Outcomes(PO) - Competencies – Performance Indicators(PI), Knowledge /Attitude Profile (WK)

PO1 : Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.				
Competency		Performance Indicators		WK
1.1	Demonstrate competence in mathematical modeling.	1.1.1	Apply the knowledge of discrete structures, linear algebra, statistics, numerical techniques and theoretical computer science to solve problems	WK1,W K2, WK3,W K4
		1.1.2	Apply the concepts of probability, statistics and queuing theory in modeling of computer based system, data and network protocols.	
1.2	Demonstrate competence in basic sciences	1.2.1	Apply laws of natural science to an engineering problem	
1.3	Demonstrate competence in engineering fundamentals	1.3.1	Apply engineering fundamentals	
1.4	Demonstrate competence in specialized engineering knowledge to the program	1.4.1	Apply theory and principles of computer science and information technology to solve an engineering problem.	
PO2 : Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)				
Competency		Performance Indicators		WK
2.1	Demonstrate an ability to identify and formulate complex engineering problem	2.1.1	Evaluate problem statements and identify objectives	WK1,W K2, WK3,W K4
		2.1.2	Identifies processes/modules/algorithms of a computer based system and parameters to solve a problem	
		2.1.3	Identify mathematical algorithmic knowledge that applies to a given problem	
2.2	Demonstrate an ability to formulate a solution plan and	2.2.1	Reframe the computer based system into interconnected subsystems	
		2.2.2	Identifies functionalities and computing resources.	

	methodology for an engineering problem	2.2.3	Identify existing solution/methods to solve the problem, including forming justified approximations and assumptions.	
		2.2.4	Compare and contrast alternative solution/methods to select the best methods.	
2.3	Demonstrate an ability to formulate and interpret a model	2.3.1	Able to apply computer engineering principles to formulate modules of a system with required applicability and performance.	
		2.3.2	Identify design constraints for required performance criteria	
2.4	Demonstrate an ability to execute a solution process and analyze results	2.4.1	Applies engineering mathematics to implement the solution.	
		2.4.2	Analyze and interpret the results using contemporary tools.	
		2.4.3	Identify the limitations of the solution and sources/causes.	
		2.4.4	Arrive at conclusions with respect to the objectives.	
PO 3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)				
Competency		Performance Indicators		WK
3.1	Demonstrate an ability to define a complex / open-ended problem in engineering terms	3.1.1	Able to define a precise problem statement with objectives and scope	WK5,W K6
		3.1.2	Able to identify and document system requirements from stake holders.	
		3.1.3	Ability to review state of the art literature to synthesize system requirements.	
		3.1.4	Ability to choose appropriate quality attributes as defined by ISO/IEC/IEEE standard.	
		3.1.5	Explore and synthesize system requirements from larger social and professional concerns. Ability to develop software requirement specifications (SRS).	
		3.1.6	Determine design, objectives, functional requirements and arrive at specifications	
3.2		3.2.1	Ability to explore design alternatives.	

	Demonstrate an ability to generate a diverse set of alternative design solutions	3.2.2	Ability to produce a variety of potential design solutions suited to meet functional requirements.	
		3.2.3	Identify suitable non functional requirements for evaluation of alternate design solutions	
3.3	Demonstrate an ability to select optimal design scheme for further development	3.3.1	Ability to perform systematic evaluation of the degree to which several design concepts meet the criteria	
		3.3.2	Consult with domain experts and stakeholders to select candidate engineering design solution for further development	
3.4	Demonstrate an ability to advance an engineering design to defined end state	3.4.1	Ability to refine architecture design into a detailed design within the existing constraints.	
		3.4.2	Ability to implement and integrate the modules.	
		3.4.3	Ability to verify the functionalities and validate the design.	
PO 4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).				
Competency		Performance Indicators		WK
4.1	Demonstrate an ability to conduct investigations of technical issues consistent with their level of knowledge and understanding	4.1.1	Define a problem for purposes of investigation, its scope and importance	WK8
		4.1.2	Ability to choose appropriate procedure/algorithm, data set and test cases.	
		4.1.3	Ability to choose appropriate hardware/software tools to conduct the experiment	
4.2	Demonstrate an ability to design experiments to solve open ended problems	4.2.1	Design and develop appropriate procedures/methodologies based on the study objectives	
4.3	Demonstrate an ability to analyze data and reach a valid conclusion	4.3.1	Use appropriate procedures, tools and techniques to collect and analyze data	
		4.3.2	Critically analyze data for trends and correlations, stating possible errors and limitations.	
		4.3.3	Represent data (in tabular and/or graphical forms) so as to facilitate analysis and explanation of the data, and drawing of conclusions	

		4.3.4	Synthesize information and knowledge about the problem from the raw data to reach appropriate conclusions	
PO 5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)				
Competency		Performance Indicators		WK
5.1	Demonstrate an ability to identify / create modern engineering tools, techniques and resources	5.1.1	Identify modern engineering tools, techniques and resources for engineering activities	WK2, WK6
		5.1.2	Create/adapt/modify/extend tools and techniques to solve engineering problems	
5.2	Demonstrate an ability to select and apply discipline specific tools, techniques and resources	5.2.1	Identify the strengths and limitations of tools for (i) acquiring information, (ii) modeling and simulating, (iii) monitoring system performance, and (iv) creating engineering designs.	
		5.2.2	Demonstrate proficiency in using discipline specific tools	
5.3	Demonstrate an ability to evaluate the suitability and limitations of tools used to solve an engineering problem	5.3.1	Discuss limitations and validate tools	
		5.3.2	Verify the credibility of results from tool use with reference to the accuracy and limitations	
PO 6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).				
Competency		Performance Indicators		WK
6.1	Demonstrate an ability to describe engineering roles in a broader context, e.g. pertaining to the environment, health, safety, legal and public welfare	6.1.1	Identify and describe various engineering roles; particularly as pertains to protection of the public and public interest at global, regional and local level	WK1,WK5, WK7

6.2	Demonstrate an understanding of professional engineering regulations, legislation and standards	6.2.1	Interpret legislation, regulations, codes, and standards relevant to your discipline and explain its contribution to the protection of the public	
6.3	Demonstrate an understanding of the impact of engineering and industrial practices on social, environmental and in economic contexts	6.3.1	Identify risks/impacts in the life-cycle of an engineering product or activity	
		6.3.2	Understand the relationship between the technical, socio economic and environmental dimensions of sustainability	
6.4	Demonstrate an ability to apply principles of sustainable design and development	6.4.1	Describe management techniques for sustainable development	
		6.4.2	Apply principles of preventive engineering and sustainable development to an engineering activity or product relevant to the discipline	
PO 7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)				
Competency		Performance Indicators		WK
7.1	Demonstrate an ability to recognize ethical dilemmas	7.1.1	Identify situations of unethical professional conduct and propose ethical alternatives	WK9
7.2	Demonstrate an ability to apply the Code of Ethics	7.2.1	Identify tenets of code of ethics given by the professional bodies like IEEE.	
		7.2.2	Examine and apply moral & ethical principles to known case studies	
PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.				
Competency		Performance Indicators		WK
8.1	Demonstrate an ability to form a team and define a role for each member	8.1.1	Recognize a variety of working and learning preferences; appreciate the value of diversity on a team	
		8.1.2	Implement the norms of practice (e.g. rules, roles, charters, agendas, etc.) of effective team work, to accomplish a goal.	
8.2	Demonstrate effective individual and team	8.2.1	Demonstrate effective communication, problem solving, conflict resolution and leadership skills	

	operations- - communication, problem solving, conflict resolution and leadership skills	8.2.2	Treat other team members respectfully
		8.2.3	Listen to other members in difficult situations
		8.2.4	Maintain composure in difficult situations
8.3	Demonstrate success in a team based project	8.3.1	Present results as a team, with smooth integration of contributions from all individual efforts

PO 9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

Competency		Performance Indicators		WK
9.1	Demonstrate an ability to comprehend technical literature and document project work	9.1.1	Read, understand and interpret technical and non- technical information	
		9.1.2	Produce clear, well-constructed, and well supported written engineering documents	
		9.1.3	Create flow in a document or presentation – a logical progression of ideas so that the main point is clear	
9.2	Demonstrate competence in listening, speaking, and presentation	9.2.1	Listen to and comprehend information, instructions, and viewpoints of others	
		9.2.2	Deliver effective oral presentations to technical and non- technical audiences	
9.3	Demonstrate the ability to integrate different modes of communication	9.3.1	Create engineering-standard figures, reports and drawings to complement writing and presentations	
		9.3.2	Use a variety of media effectively to convey a message in a document or a presentation	

PO 10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

Competency		Performance Indicators		WK
10.1	Demonstrate an ability to evaluate the economic and financial performance of an engineering activity	10.1.1	Describe various economic and financial costs/benefits of an engineering activity	
		10.1.2	Analyze different forms of financial statements to evaluate the financial status of an engineering project	

10.2	Demonstrate an ability to compare and contrast the costs/benefits of alternate proposals for an engineering activity	10.2.1	Analyze and select the most appropriate proposal based on economic and financial considerations.	
10.3	Demonstrate an ability to plan/manage an engineering activity within time and budget constraints	10.3.1	Identify the tasks required to complete an engineering activity, and the resources required to complete the tasks.	
		10.3.2	Use project management tools to schedule an engineering project so it is completed on time and on budget.	
PO 11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.				
Competency		Performance Indicators		WK
11.1	Demonstrate an ability to identify gaps in knowledge and a strategy to close these gaps	11.1.1	Describe the rationale for requirement for continuing professional development	WK8
		11.1.2	Identify deficiencies or gaps in knowledge and demonstrate an ability to source information to close this gap	
11.2	Demonstrate an ability to identify changing trends in engineering knowledge and practice	11.2.1	Identify historic points of technological advance in engineering that required practitioners to seek education in order to stay current	
		11.2.2	Recognize the need and be able to clearly explain why it is vitally important to keep current regarding new developments in your field	
11.3	Demonstrate an ability to identify and access sources for new information	11.3.1	Source and comprehend technical literature and other credible sources of information	
		11.3.2	Analyze sourced technical and popular information for feasibility, viability, sustainability, etc.	

PSO1: Apply the fundamentals of science, engineering and mathematics to understand, analyze and develop solutions in the areas related to artificial intelligence and data science for optimal design of intelligent systems.

Competency	Performance Indicators	WK
-------------------	-------------------------------	-----------

1.1	Ability to apply linear algebra, calculus, probability, and statistics to analyze data and model AI/ML systems.	1.1.1	Applies linear algebra, calculus, probability, and statistics to solve AI/DS-related problems.	WK1, WK2, WK3, WK4, WK9
		1.1.2	Interprets mathematical results and uses them to guide AI model development.	
1.2	Ability to formulate real-world problems into computational AI/DS problems and design suitable algorithms.	1.2.1	Identifies real-world problems and formulates them into AI/DS tasks.	
		1.2.2	Selects and justifies suitable algorithms for the implementation of defined computational tasks.	
1.3	Ability to build and optimize intelligent systems using engineering principles and scientific methodologies.	1.3.1	Develops AI/ML models using scientific principles and systematic approaches.	
		1.3.2	Optimizes model performance through experimentation and parameter tuning.	

PSO2: Design and implement appropriate techniques and analytic tools for the integration of intelligent systems, with a view to engaging in lifelong learning for the betterment of society.

Competency		Performance Indicators		WK
2.1	Ability to select and use modern AI/DS tools, frameworks, and platforms to design and implement intelligent systems.	2.1.1	Demonstrates proficiency with AI/DS tools (Python, TensorFlow, PyTorch, SQL, etc.).	WK3, WK4, WK5, WK6, WK9
		2.1.2	Implements data preprocessing, feature engineering, and model building using tools.	
2.2	Ability to integrate data pipelines, models, and computing architectures into deployable intelligent systems that address real-world societal needs.	2.2.1	Integrates data pipelines, models, and APIs into functional systems and deploys on cloud/edge/embedded platforms that support applications with societal relevance.	
		2.2.2	Tests and validates deployed systems for reliability, scalability, and performance.	

2.3	Ability to independently learn emerging technologies, paradigms, and tools to remain current in AI and data science.	2.3.1	Demonstrates ability to learn and use emerging AI tools/frameworks independently.	
		2.3.2	Evaluates new methodologies and compares them with existing approaches.	
PSO3: Practice professional ethics in applying scientific method to model and support multidisciplinary facets of engineering and its societal implications.				
Competency		Performance Indicators		WK
3.1	Ability to evaluate and apply ethical principles, fairness, privacy, and transparency in AI systems.	3.1.1	Applies ethical guidelines and responsible AI principles in solution design.	WK1, WK3, WK6, WK7, WK8, WK9
		3.1.2	Evaluates AI systems for compliance with societal and regulatory standards.	
3.2	Ability to apply scientific methodology—hypothesis, experimentation, validation—to solve multidisciplinary engineering problems.	3.2.1	Uses analytical and experimental methods to validate models and systems.	
		3.2.3	Interprets experimental results and draws evidence-based conclusions.	
3.3	Ability to analyze the societal, environmental, and interdisciplinary implications of AI/DS technologies.	3.3.1	Assesses societal and environmental impacts of AI/DS technologies.	
		3.3.2	Works effectively in multidisciplinary teams to design solutions considering societal needs.	

COURSE OUTCOMES:

Si.NO	DESCRIPTION	Blooms' Taxonomy Level
CO1	Model and solve real world problems by applying knowledge across domains	Level 3: Apply
CO2	Develop products, processes or technologies for sustainable and socially relevant applications	Level 3: Apply
CO3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks	Level 3: Apply
CO4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms	Level 3: Apply
CO5	Identify technology/research gaps and propose innovative/creative solutions	Level 4: Analyze
CO6	Organize and communicate technical and scientific findings effectively in written and oral forms	Level 3: Apply

CO-PO AND CO-PSO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	2	2	2	1	2	2	1	1	1	1	2
CO2	2	2	2		1	3	1	1		1	1
CO3								3	2	2	1
CO4					2		3	2	2	3	2
CO5	2	3	3	1	2						1
CO6					2		2	2	3	1	1

	PSO1	PSO2	PSO3
CO1	3	3	3
CO2	3		3

CO3	2	2	2
CO4	3	3	3
CO5	2		2
CO6			3

Mapping with Justification

CO1: Model and solve real world problems by applying knowledge across domains					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO1	1.1 – Demonstrate competence in mathematical modeling	1.1.1, 1.3.1	2	WK1–WK4	Students apply engineering fundamentals and mathematical reasoning while identifying and solving real-world problems.
PO2	2.1 – Ability to identify and formulate complex engineering problems	2.1.1, 2.1.2	2	WK1–WK4	Students analyze problem statements, define objectives, choose parameters, and justify analytical approaches.
PO3	3.1 – Ability to define a complex/open problem	3.1.1	2	WK5–WK6	Students refine open-ended real-world needs into formal design statements and computational objectives.
PO4	4.3 – Ability to analyze data and reach conclusions	4.3.1	1	WK8	Students interpret analytical results but detailed experimental investigation is limited.
PO5	5.2 – Selection and application of tools	5.2.1	2	WK2, WK6	Students use IT/engineering tools for modeling simulations or exploratory analysis.
PO6	6.3 – Understanding societal dimensions	6.3.1	2	WK1, WK5, WK7	Real-world problem solving requires basic understanding of societal constraints and sustainability.

CO1: Model and solve real world problems by applying knowledge across domains					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO7	7.1 – Ethical alternatives	7.1.1	1	WK9	Ethical considerations are peripheral, hence low mapping.
PO8	8.2 – Individual/team operations	8.2.1	1	—	Some collaborative interpretation and division of analytical work occurs.
PO9	9.1 – Technical comprehension and reporting	9.1.2	1	—	Limited reporting of analytical results.
PO10	10.3 – Engineering activity planning	10.3.1	1	—	Minimal planning or scheduling required in CO1.
PO11	11.1 – Identifying gaps and sourcing information	11.1.2	2	WK8	Students research domain concepts, benchmark solutions, and learn independently.
PSO1	1.2 – Formulation of AI/DS problems	1.2.1, 1.3.1	3	WK1–WK4, WK9	Strong mapping because real-world data analysis, algorithm design, and model formulation directly align with PSO1.
PSO2	2.1 – Tool-based implementation	2.1.2	3	WK3–WK6	Students apply analytic and computational tools for solving domain problems.
PSO3	3.2 – Evidence-based conclusions	3.2.3	3	WK1, WK3, WK6–WK9	Students justify conclusions, interpret outcomes, and relate results to real-world societal and interdisciplinary constraints.

CO2: Develop products, processes or technologies for sustainable and socially relevant applications					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO1	1.1 – Mathematical reasoning	1.1.1	2	WK1–WK4	Engineering fundamentals support sustainable design analysis.
PO2	2.1 – Formulate system requirements	2.1.2	2	WK1–WK4	Students identify modules, processes, architecture, and constraints.
PO3	3.1 – Design complex solutions	3.1.6, 3.3.1	2	WK5–WK6	Translating sustainable needs into design specifications aligns with PO3.
PO6	6.3 – Sustainability awareness	6.3.2	1	WK1, WK5, WK7	Sustainability considerations are relevant but not deeply analytical in CO2.
PO7	7.1 – Ethical responsibility	7.1.1	3	WK9	Strong emphasis on societal responsibility, safety and ethical relevance in sustainable design.
PO8	8.2 – Team coordination	8.2.1	1	—	Limited but valid team communication.
PO9	9.1 – Documentation	9.1.2	1	—	Minimal reporting during development.
PO11	11.3 – Lifelong learning	11.3.1	1	WK8	Basic requirement for researching sustainable standards and emerging technologies.
PSO1	1.3 – Build and optimize intelligent systems	1.3.1	3	WK1–WK4, WK9	Strong because sustainable solutions involve scientific design, experimentation and optimization.

PSO3	3.3 – Societal implications	3.3.1	3	WK1, WK3, WK6–WK9	Strong mapping because designs must consider ethics, safety, and multidisciplinary societal needs.
------	-----------------------------	-------	---	-------------------	--

CO3: Function effectively as an individual and as a leader in diverse teams					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO8	8.3 – Demonstrate success in a team-based project	8.3.1	3	—	CO3 directly focuses on collaborative execution, leadership, conflict resolution, and task integration.
PO9	9.2 – Listening, speaking and presentation	9.2.1	2	—	Effective team performance requires oral communication and idea exchange.
PO10	10.3 – Resource/task scheduling	10.3.1	2	—	Team leaders must define tasks, roles and micro-schedules.
PO11	11.1 – Sourcing new information	11.1.1	1	WK8	Minimal but valid contribution to information searching for teamwork.
PSO1	1.3 – System execution in teams	1.3.2	2	WK1–WK4, WK9	Students jointly execute experiments and interpret observations in teams.
PSO2	2.1 – Collaborative tool usage	2.1.1	2	WK3–WK6	Teams use collaborative platforms, notebooks, repositories and analytic tools.
PSO3	3.3 – Multidisciplinary collaboration	3.3.2	2	WK1, WK3, WK6–WK9	Working across disciplines aligns with professional ethics, societal sensitivity and multi-domain reasoning.

CO4: Plan and execute tasks utilizing available resources and follow ethical norms					
PO/PSO	Competency	Selected Pls	Strength	WK	Justification
PO3	3.4 – Ability to refine and implement design	3.4.1	2	WK5–WK6	CO4 requires planning, structured execution and iterative refinement of system design.
PO6	6.3 – Sustainability awareness	6.3.1	3	WK1, WK5, WK7	Strong because planning and execution reflect societal, safety and sustainability concerns.
PO7	7.1 – Professional ethics	7.1.1	2	WK9	Ethical use of resources, authorship, safety, and professional practice directly support CO4.
PO8	8.3 – Team execution	8.3.1	2	—	Execution of shared tasks requires coordinated teamwork.
PO9	9.1 – Structured documentation	9.1.2	3	—	Strong mapping because execution must be documented professionally with clarity.
PO10	10.3 – Project scheduling	10.3.2	2	—	Resource allocation, Gantt-charts, and milestone planning support CO4.
PSO1	1.3 – Build and optimize intelligent systems	1.3.2	3	WK1–WK4, WK9	Strong because CO4 requires systematic execution and validation of intelligent system design.
PSO2	2.2 – Deploy and validate systems	2.2.2	3	WK3–WK6	Strong alignment with execution, deployment, verification and performance tracking.
PSO3	3.2 – Scientific method and ethical validation	3.2.1	3	WK1, WK3, WK6–WK9	Execution requires transparent validation, evidence-based analysis, and ethical reporting.

CO5: Identify research gaps and propose innovative/creative solutions					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO1	1.1 – Mathematical modeling	1.1.1	2	WK1–WK4	Research gap analysis partially depends on mathematical and scientific reasoning.
PO2	2.3 – Formulate and interpret models	2.3.2	3	WK1–WK4	Strong because students synthesize alternatives, evaluate literature and refine hypotheses.
PO3	3.2 – Explore design alternatives	3.2.1	3	WK5–WK6	Proposing new solutions aligns with design innovation and exploration.
PO4	4.3 – Investigation and inference	4.3.3	1	WK8	Limited experimental investigation.
PO5	5.2 – Tool adoption	5.2.2	2	WK2, WK6	Students perform exploratory analysis, benchmarking and model comparisons using analytical tools.
PO11	11.2 – Trend awareness & lifelong learning	11.2.2	1	WK8	Minimal but valid contribution where students scan emerging technologies.
PSO1	1.2 – Algorithm selection & justification	1.2.2	2	WK1–WK4, WK9	Moderate because gap-based innovation drives algorithmic improvements.
PSO3	3.3 – Societal implications	3.3.1	2	WK1, WK3, WK6–WK9	Students consider societal or ethical dimensions of proposed solutions.

CO6: Organize and communicate technical and scientific findings					
PO/PSO	Competency	Selected PIs	Strength	WK	Justification
PO5	5.2 – Select and apply discipline-specific tools	5.2.1, 5.2.2	2	WK2, WK6	Students use analysis/reporting tools to prepare technical results and visualizations.
PO7	7.2 – Apply professional ethics	7.2.2	2	WK9	Ethical reporting, citation, attribution and transparency in communication.
PO8	8.3 – Team-based presentation	8.3.1	2	—	Students prepare and present integrated findings collaboratively.
PO9	9.1 & 9.3 – Technical reporting and multimodal communication	9.1.2, 9.3.2	3	—	Strong mapping because CO6 directly evaluates structured documentation, clarity, and professional scientific communication.
PO10	10.3 – Task/communication planning	10.3.1	1	—	Minimal but valid contribution: communication artifacts require planning and scheduling.
PO11	11.3 – Access sources and information	11.3.1	1	WK8	Students read and synthesize credible sources to support communication.
PSO3	3.2 & 3.3 – Scientific validation & societal implications	3.2.3, 3.3.1	3	WK1, WK3, WK6, WK7, WK8, WK9	Strong mapping because CO6 enforces evidence-based reasoning, interpretation, scientific reporting and ethical societal framing.